IT5022 – Fundamentals of Machine Learning

Group Assignment 01

Machine Learning-Based Prediction of Electrical Grid Stability

A comparison of Logistic Regression and Random Forest on the UCI Electrical Grid Stability Simulated Data

Table 1. Group members

Member	Name	Student ID	Email
Member 1	Charuka Kavinda	[Student ID]	ckavinda57@gmail.com
Member 2	Nadeera Rajapaksha	[Student ID]	[Email]

GitHub: https://github.com/Ckavinda98/IT5022_Fundamentals_of_Machine_Learning_Assignment-

Dataset: https://doi.org/10.24432/C5PG66

Abstract

Decentralised smart grids adjust electricity prices to grid frequency so that producers and consumers respond to supply–demand imbalance. Whether such a feedback loop remains stable depends on how quickly participants react and how strongly they respond to price. This study asks whether the operating parameters of a simulated four-node grid can predict if an operating point is stable. Using the 10,000-record UCI Electrical Grid Stability Simulated Data, we removed the continuous stability score stab after showing that the label stabf is an exact threshold of it, and trained Logistic Regression and Random Forest on the 12 remaining operating parameters with a stratified 80/20 split. On the held-out test set, Random Forest (92.9% accuracy, macro-F1 0.922) clearly outperformed linear Logistic Regression (82.0%, 0.801) and the 63.8% majority baseline. Cross-validated experiments on the training set then showed that the linear model's weakness came from its features rather than the algorithm: adding degree-2 interaction terms raised Logistic Regression to 95.6% test accuracy (macro-F1 0.952), the best result in the study, while tuning Random Forest gave no gain. Feature ablation and permutation importance showed that reaction times and price elasticities carry nearly all the predictive signal and that the producer power p1 is fully redundant. Lowering the decision threshold reduced missed instabilities from 45 to 17 of 1,276 unstable test cases at the cost of more false alarms. Because the data are simulated from a single idealised topology, the results demonstrate learnability, not operational safety.

1. Introduction

1.1 Background and problem

Growing shares of renewable generation make electricity supply more variable, and grid operators increasingly rely on demand-side flexibility to keep supply and demand balanced. In the Decentral Smart Grid Control (DSGC) concept, the electricity price is tied to the local grid frequency: when frequency drops (demand exceeds supply) the price rises and participants reduce consumption, and vice versa (Schäfer et al., 2016). This avoids a central communication layer, but it introduces a feedback loop with delays. If participants react too slowly or too strongly, the loop can amplify rather than damp disturbances, and the grid becomes unstable.

Checking stability exactly requires solving the system's differential equations and examining the roots of its characteristic equation for each operating point. A data-driven classifier that maps operating parameters directly to a stability label could act as a fast screening tool and could show which parameters drive instability.

1.2 Research question and objectives

Research question: Can the operating parameters of a simulated decentralised electrical grid (reaction times, nominal powers and price elasticities) reliably predict whether an operating point is stable or unstable, and which learning approach captures that relationship best?

- Audit the dataset and identify cleansing, validity and target-leakage issues.
- Train two distinct, non-deep-learning supervised algorithms — Logistic Regression and Random Forest — on identical, leakage-free data.

- Compare them against a majority baseline using accuracy, class-wise precision/recall/F1, macro-F1, ROC-AUC and confusion matrices, with attention to the cost of missed instabilities.
- Investigate how accuracy can be improved through feature engineering, hyperparameter tuning, feature selection and decision-threshold adjustment, using cross-validation only.
- Interpret which parameters drive stability and discuss limitations of simulated data.

2. Dataset

2.1 Source and context

The study uses the **Electrical Grid Stability Simulated Data** donated to the UCI Machine Learning Repository by Vadim Arzamasov in November 2018 (Arzamasov, 2018; DOI 10.24432/C5PG66), released under the Creative Commons Attribution 4.0 licence. It was produced for research on concise stability models of the DSGC system (Arzamasov, Böhm and Jochem, 2018). The copy analysed here was obtained from its Kaggle mirror (https://www.kaggle.com/datasets/avikdas567/electrical-grid-stability-and-control-simulation) and matches the UCI description: 10,000 records, 12 input features and two target representations.

Each record is one simulated configuration of a four-node star network: one producer (node 1) connected to three consumers (nodes 2–4). For each configuration the authors sampled the parameters, solved the linearised system, and recorded the largest real part of the characteristic equation's roots. A negative value means disturbances die out (stable); a positive value means they grow (unstable).

2.2 Justification of dataset choice

The dataset is published by a credible repository, describes a real engineering problem, and is cited in peer-reviewed work. It is complex enough for a meaningful study: no single feature separates the classes, the true decision boundary is non-linear, and a linear model reaches only 82% accuracy. It also contains non-obvious issues — a leaking target column and an exactly redundant feature — that must be found and handled. The dataset is popular, so rather than reproducing a standard tutorial this study builds an independent analysis: a formal leakage audit, a redundancy proof, feature-group ablation, interaction-feature engineering, and a safety-oriented evaluation of missed instabilities. The data are simulated from physical equations rather than measured on a live grid; this is discussed as a limitation in Section 8.

2.3 Attributes

Table 2. Dataset attributes (definitions and ranges from the UCI documentation and observed data)

Attribute	Description	Role / type	Range
tau1	Reaction time of the producer	Input, continuous	0.5 – 10 s
tau2 – tau4	Reaction times of consumers 1–3	Input, continuous	0.5 – 10 s
p1	Nominal power produced (positive); p1 = −(p2 + p3 + p4)	Input, continuous	1.58 – 5.86 (observed)
p2 – p4	Nominal power consumed (negative)	Input, continuous	-2.0 – -0.5 s^{-2}
g1	Price-elasticity coefficient (γ) of the producer	Input, continuous	0.05 – 1.0 s^{-1}

g2 – g4	Price-elasticity coefficients of consumers 1–3	Input, continuous	0.05 – 1.0 s^{-1}
stab	Maximal real part of the characteristic equation root	Continuous score (excluded)	−0.081 – 0.109
stabf	Stability label: 'stable' or 'unstable'	Target, binary	2 classes

2.4 Size and class balance

The dataset has 10,000 rows and 14 columns (about 2 MB), all numeric except the text label. There are 6,380 unstable (63.8%) and 3,620 stable (36.2%) records. A classifier that always predicts 'unstable' therefore scores 63.8% accuracy; this majority (zero-rule) baseline is the minimum a useful model must beat, and the imbalance means accuracy must be reported together with class-wise metrics.

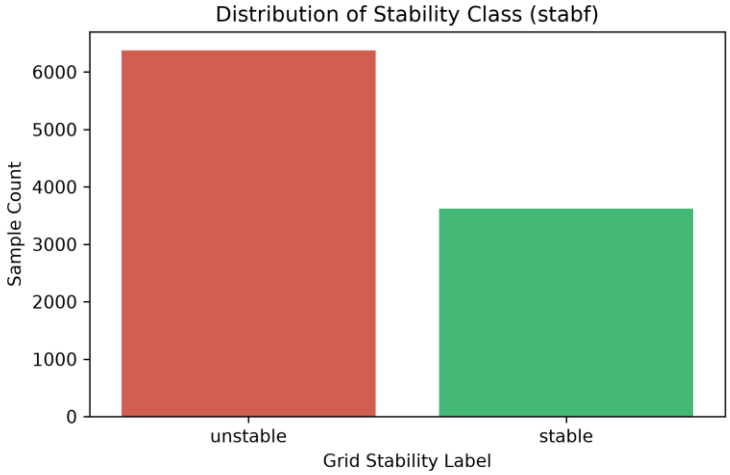


Figure 1. Class distribution of stabf.

3. Exploratory Data Analysis and Preprocessing

3.1 Data audit and cleansing

The audit (notebook 01, src/data_checks.py) found **no missing values, no duplicated records and no duplicated feature vectors**, so no imputation or de-duplication was needed. Each feature was then checked against its documented generation range, and the data were checked against the physical power-balance constraint and the labelling rule.

Table 3. Validity checks on all 10,000 records

Validity check	Violating rows
tau1–tau4 within [0.5, 10]	0
p2–p4 within [−2, −0.5]	0
g1–g4 within [0.05, 1]	0
p1 + p2 + p3 + p4 = 0 (max |sum| = 5.8×10^{-15})	0
stabf = 'stable' exactly when stab ≤ 0	0

stabf only contains 'stable' / 'unstable'	0

All values are valid and in range, so no records or outliers were removed. Two checks have important consequences: the label is a deterministic function of stab, and p1 is an exact linear combination of p2–p4.

3.2 Target leakage: removal of stab

Figure 2 shows that the two classes occupy completely separate ranges of stab: every stable record has stab ≤ 0 (maximum −0.000002) and every unstable record has stab > 0 (minimum 2.6×10^{-7}). The label stabf is simply the sign of stab. A model given stab would reach ~100% accuracy by learning one threshold, which would answer nothing about whether the operating parameters predict stability. stab was therefore **excluded from the feature set in every experiment**, leaving X = {tau1–tau4, p1–p4, g1–g4} and y = stabf.

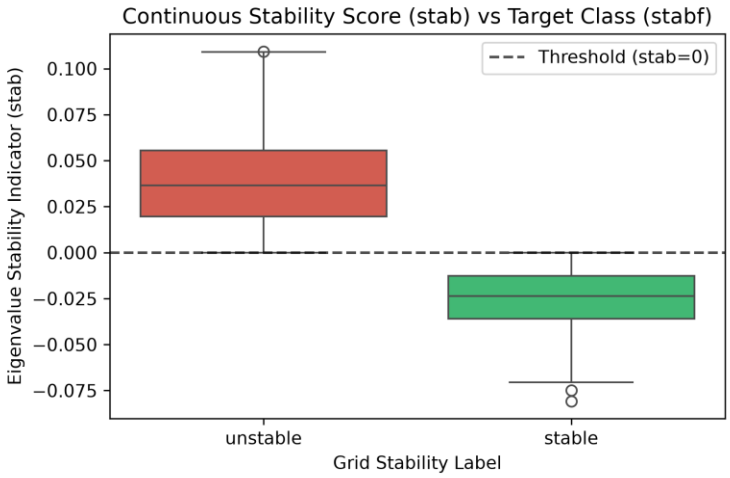


Figure 2. stab by stabf. The dashed line at stab = 0 separates the classes perfectly, demonstrating target leakage.

3.3 Correlations and class-wise distributions

Almost all pairwise correlations between features are close to zero (Figure 3), consistent with independent sampling of the simulation inputs. The exceptions are the correlations of about −0.58 between p1 and each consumer power, a direct result of the power balance. Pairwise correlation understates this dependency: together, p2–p4 determine p1 exactly.

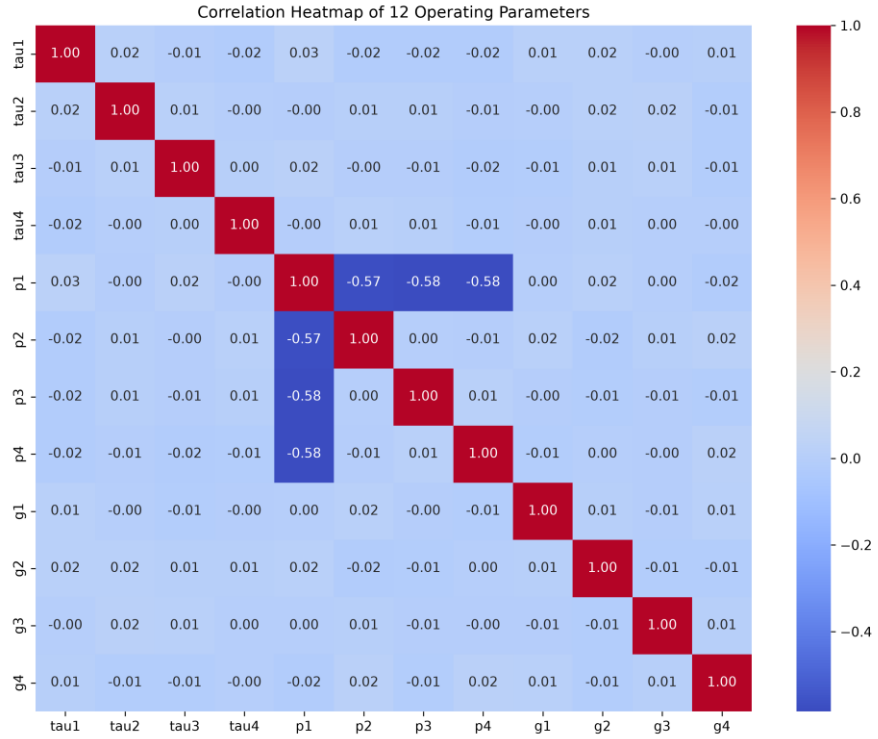


Figure 3. Correlation matrix of the 12 operating parameters.

Figure 4 compares feature distributions between classes. Unstable configurations have noticeably longer reaction times (mean tau ≈ 5.74 s vs ≈ 4.38 s, about 0.5 standard deviations higher) and higher price elasticities (mean g ≈ 0.57 vs ≈ 0.45). The power distributions are almost identical for both classes (standardised difference < 0.05). Slow, strongly price-reactive participants therefore make the feedback loop oscillate. However, the class distributions overlap heavily for every single feature, so stability depends on combinations of parameters — a first indication that a non-linear decision boundary will be needed.

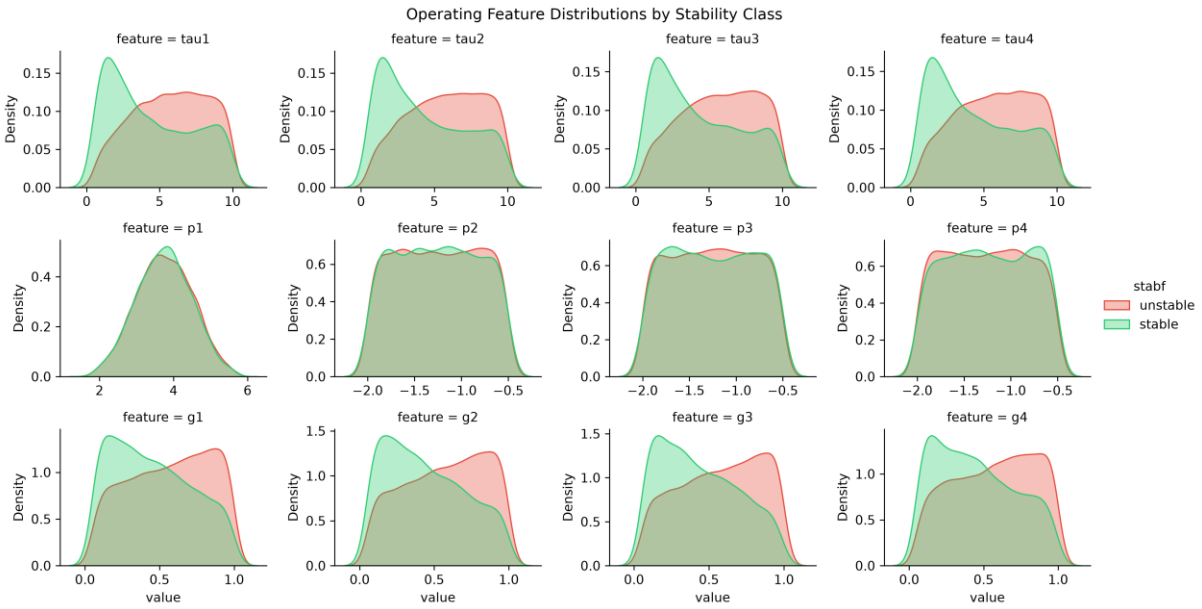


Figure 4. Density of each feature by stability class.

3.4 Train/test split and scaling

The data were split once into 8,000 training and 2,000 test records using a **stratified 80/20 split** (random_state = 42), which preserves the 63.8/36.2 class ratio in both sets. The test set was used only for final evaluation; every modelling decision in Section 7 was made with stratified 5-fold cross-validation inside the training set. For Logistic Regression, features were standardised with StandardScaler placed inside a scikit-learn Pipeline, so the scaling parameters are learned only from training data (or training folds) and no test information leaks into preprocessing. Random Forest is invariant to monotonic feature scaling and was trained on raw values.

3.5 Dimensionality reduction

Principal Component Analysis on the standardised training features (Figure 5) shows that the twelfth component explains exactly 0% of the variance: the 12 features span only 11 dimensions because of the power balance. The remaining components each explain roughly 8–9% of the variance (the first, about 17%, captures the shared power variation), so no small subset of components preserves most of the information. PCA would discard signal and make the model harder to interpret without simplifying the problem, so it was not used. The appropriate reduction is to remove the redundant p1; Section 7.2 tests this and wider feature-group removal with cross-validation.

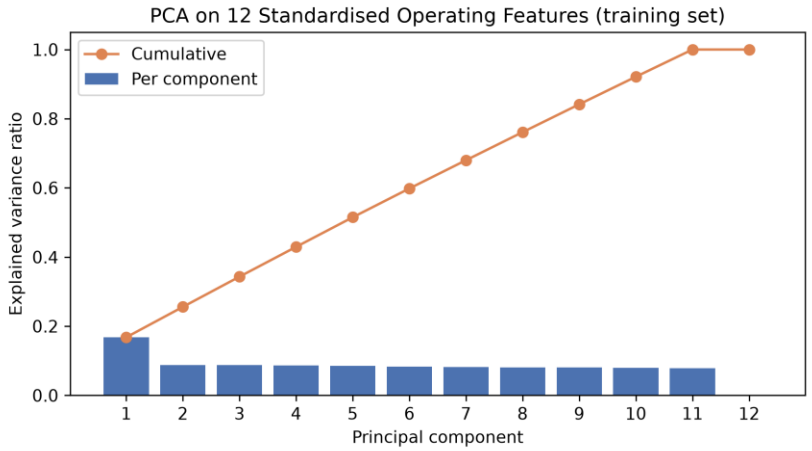


Figure 5. PCA explained variance of the standardised training features.

4. Learning Algorithms

The task is supervised binary classification, so two distinct supervised algorithms were applied, as required for a two-person team. Both are classical (non-deep-learning) methods that represent contrasting modelling assumptions: a parametric linear model and a non-parametric tree ensemble.

4.1 Logistic Regression

Logistic Regression (Cox, 1958) models the probability of the positive class (here 'unstable') as the logistic (sigmoid) function of a linear combination of the inputs:

$$P(\text{unstable} \mid x) = 1 / (1 + \exp(-(\beta_0 + \beta_1 x_1 + \ldots + \beta_{12} x_{12})))$$

Equivalently, the log-odds are linear in the features, so the decision boundary P = 0.5 is a hyperplane. The coefficients are estimated by minimising the log-loss (maximising the likelihood), and scikit-learn adds an L2 penalty controlled by C (smaller C = stronger regularisation) to prevent large, unstable coefficients (Hastie, Tibshirani and Friedman, 2009). Because the penalty treats all coefficients equally, features must be on comparable scales, hence standardisation.

Justification: Logistic Regression is fast, well understood, produces calibrated probabilities and has directly interpretable coefficients (each standardised coefficient is the change in log-odds per standard deviation). It is the natural baseline for binary classification and tests whether stability can be explained by a linear combination of parameters. Its main limitation is that it cannot represent interactions between features unless they are added explicitly; Section 7.3 does this with polynomial features.

Settings: StandardScaler → LogisticRegression(C = 1.0, L2 penalty, lbfgs solver, max_iter = 2000, random_state = 42). The model converged without warnings.

4.2 Random Forest

A Random Forest (Breiman, 2001) is an ensemble of decision trees. A decision tree recursively splits the feature space with axis-aligned thresholds, choosing at each node the split that most reduces Gini impurity, until leaves are pure. Single deep trees fit the training data closely but have high variance. Random Forest reduces this variance in two ways: each tree is trained on a bootstrap sample of the training records (bagging), and each split considers only a random subset of features (here $\sqrt{12} \approx 3$), which decorrelates the trees. Class probabilities are averaged across trees and the class with the higher average is predicted.

Justification: EDA showed that no single feature separates the classes and that stability depends on combinations of parameters. Random Forest models such non-linear interactions automatically, needs no scaling, is robust to redundant features, and provides feature-importance estimates. It therefore contrasts directly with the linear baseline. Its limitations are lower interpretability, a tendency to overfit with fully grown trees, and an axis-aligned, step-shaped decision boundary.

Settings: RandomForestClassifier(n_estimators = 200, criterion = 'gini', max_features = 'sqrt', max_depth = None, min_samples_leaf = 1, random_state = 42).

4.3 Comparison of the two approaches

Table 4. Contrast between the two algorithms

Aspect	Logistic Regression	Random Forest
Model family	Parametric, linear in log-odds	Non-parametric ensemble of trees
Decision boundary	Single hyperplane	Piecewise, axis-aligned, non-linear
Feature interactions	Only if engineered explicitly	Learned automatically
Scaling needed	Yes	No
Main error source	Bias (too simple)	Variance (overfitting)
Interpretability	High (coefficients)	Moderate (importance scores)
Training cost	Very low	Moderate

5. Experimental Setup and Evaluation Metrics

- **Data:** 12 operating features; stab excluded; stratified 80/20 split (8,000 / 2,000), random_state = 42.
- **Validation:** stratified 5-fold cross-validation (shuffled, random_state = 42) on the training set for model assessment, feature ablation, hyperparameter tuning and threshold selection.
- **Test protocol:** the test set was used once, after all choices were fixed, for every model reported.
- **Baseline:** majority (zero-rule) classifier that always predicts 'unstable'.
- **Tools:** Python 3.14, pandas 3.0, NumPy 2.5, scikit-learn 1.9 (Pedregosa et al., 2011), matplotlib, seaborn; code in src/ and notebooks 01–07 in the GitHub repository.

Treating 'unstable' as the positive class, the metrics used are:

Table 5. Evaluation metrics

Metric	Definition	Why it matters here
Accuracy	(TP + TN) / N	Overall correctness; compare to 63.8% baseline
Precision	TP / (TP + FP)	How many 'unstable' alarms are real
Recall	TP / (TP + FN)	How many unstable grids are caught
F1-score	2·P·R / (P + R)	Balances precision and recall per class
Macro-F1	Mean of the two class F1 scores	Treats the minority stable class equally
ROC-AUC	Area under ROC curve	Threshold-independent ranking quality
False stable	Unstable predicted stable (FN)	Most safety-critical error: missed instability
False unstable	Stable predicted unstable (FP)	False alarm: unnecessary intervention

6. Results

6.1 Held-out test performance

Table 6. Test-set results of the two algorithms with baseline settings (N = 2,000; 1,276 unstable, 724 stable)

Metric	Majority baseline	Logistic Regression	Random Forest
Accuracy	0.6380	0.8200	0.9290
Macro-F1	0.3895	0.8009	0.9220
Weighted F1	0.4970	0.8179	0.9285
Stable precision	0.0000	0.7774	0.9292
Stable recall	0.0000	0.7044	0.8702
Stable F1	0.0000	0.7391	0.8987
Unstable precision	0.6380	0.8408	0.9289
Unstable recall	1.0000	0.8856	0.9624
Unstable F1	0.7790	0.8626	0.9453

ROC-AUC	0.5000	0.8920	0.9823
False stable (missed unstable)	0	146	48
False unstable (false alarm)	724	214	94

Both algorithms clearly beat the majority baseline. Random Forest is better than Logistic Regression on every metric: accuracy rises from 82.0% to 92.9%, macro-F1 from 0.801 to 0.922, and ROC-AUC from 0.892 to 0.982. Most importantly for grid operation, Random Forest misses only 48 of the 1,276 unstable configurations (unstable recall 96.2%) compared with 146 for Logistic Regression (88.6%), and it also raises fewer false alarms (94 vs 214). For both models the minority stable class is harder to recognise (stable recall 70.4% and 87.0%).

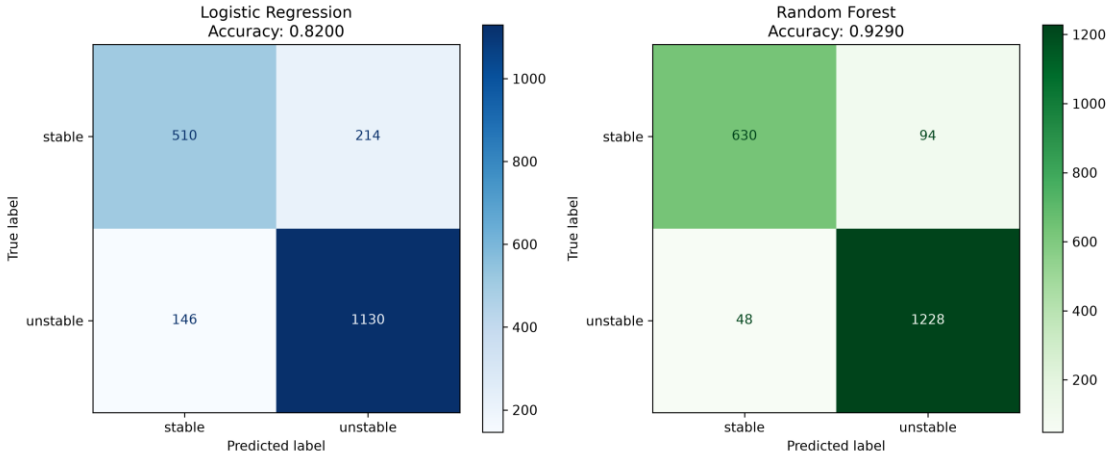


Figure 6. Confusion matrices on the test set (rows = actual, columns = predicted).

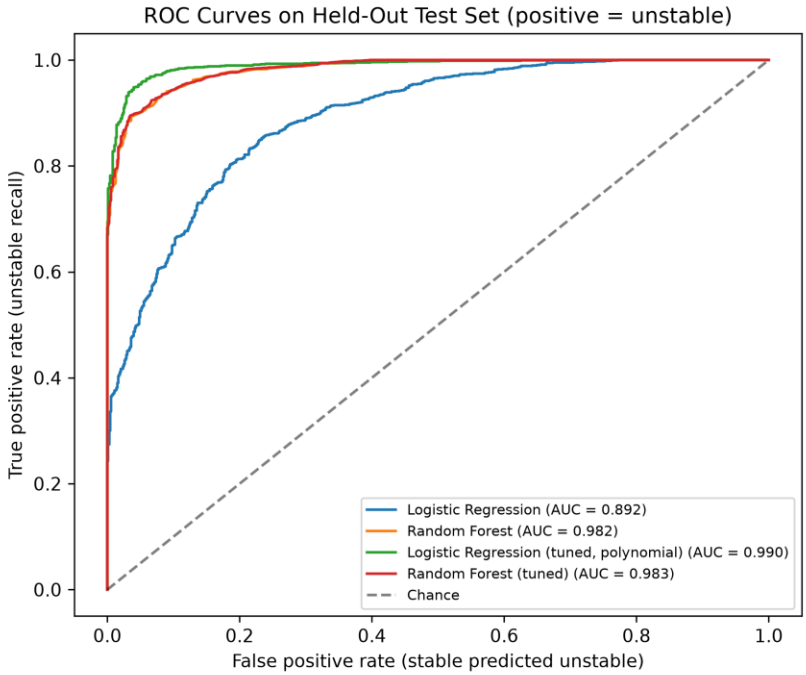


Figure 7. ROC curves on the test set for the baseline and tuned models (positive class = unstable).

6.2 Cross-validation and overfitting

Table 7. 5-fold cross-validation on the training set versus training and test accuracy

Model	CV accuracy (mean ± sd)	CV macro-F1 (mean ± sd)	CV ROC-AUC	Training accuracy	Test accuracy
Logistic Regression	0.8146 ± 0.0072	0.7956 ± 0.0080	0.8895	0.8140	0.8200
Random Forest	0.9162 ± 0.0109	0.9078 ± 0.0118	0.9777	1.0000	0.9290

Cross-validation scores vary by less than 1.2 percentage points across folds and agree with the test scores, so the comparison is not an artefact of one lucky split. Logistic Regression has almost identical training and validation accuracy (≈ 81%): it does not overfit, and its errors are due to bias — a linear boundary is too simple. Random Forest reaches 100% training accuracy but ≈ 92% in cross-validation: its fully grown trees memorise the training data, and averaging only partly cancels this variance.

6.3 Where do the errors occur?

Misclassified test records lie close to the stability boundary. For Random Forest, 61% of errors have |stab| < 0.01, although only 14% of all test records are that close to zero; the median |stab| of misclassified records is 0.0076 versus 0.0325 for correctly classified ones. For Logistic Regression the pattern is weaker (30% of errors within |stab| < 0.01; median 0.0159 vs 0.0350), because its linear boundary also misclassifies configurations far from the true boundary. Near stab = 0, tiny parameter changes flip the label, so some residual error is unavoidable for any model.

6.4 Feature importance

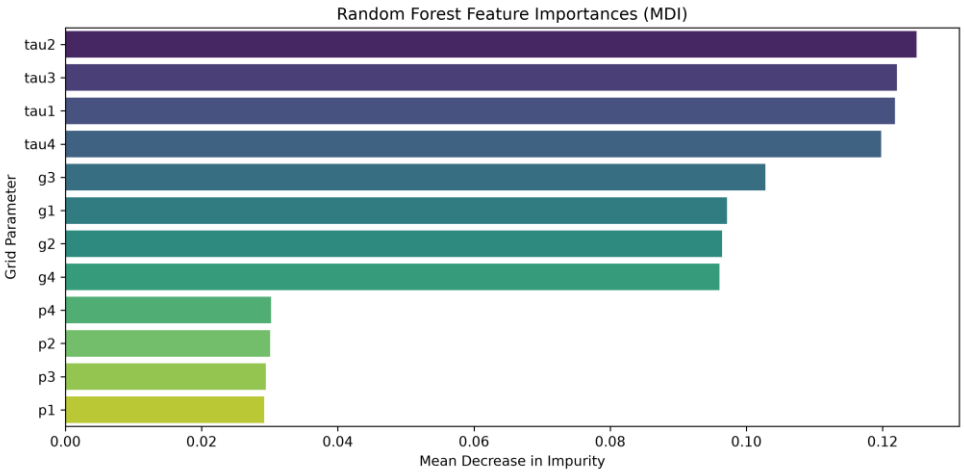


Figure 8. Random Forest impurity-based (MDI) feature importance.

Table 8. Permutation importance on the test set (mean drop in macro-F1 over 10 shuffles)

Feature	Logistic Regression	Random Forest
tau1	0.0596	0.1207
tau4	0.0678	0.1158
tau2	0.0577	0.1095

tau3	0.0537	0.1085
g2	0.0579	0.0937
g4	0.0495	0.0884
g3	0.0599	0.0883
g1	0.0461	0.0771
p1	0.0002	0.0023
p2	0.0003	0.0020
p3	0.0004	0.0020
p4	-0.0013	0.0003

Both importance measures agree. The four reaction times are the strongest predictors (MDI ≈ 0.12 each; shuffling one lowers Random Forest macro-F1 by 0.11–0.12), followed by the four price elasticities (MDI ≈ 0.10; drop 0.077–0.094). The power variables contribute almost nothing (permutation importance ≤ 0.003). Permutation importance was used to confirm the MDI ranking because MDI is known to favour continuous features with many split points (Strobl et al., 2007); here all features are continuous, and the two methods give the same ordering.

7. Improving Accuracy: Comparison and Critical Discussion

The rubric question 'how could the accuracy be improved?' was investigated with four experiments (src/experiments.py, notebook 07). All choices were made by cross-validation on the training set; the test set was used only for the final comparison in Section 7.5.

7.1 Why does Random Forest beat Logistic Regression?

The physics of the DSGC model offers an explanation. Stability depends on how a participant's reaction delay combines with its price sensitivity: a strong response that arrives late pushes the system the wrong way. This is an interaction — the effect of tau depends on g — and a model whose log-odds are a weighted sum of tau and g cannot represent it. Random Forest captures interactions through successive splits on different features, which explains its 11-point accuracy advantage. This suggested a testable hypothesis: if interaction terms are supplied, Logistic Regression should close the gap.

7.2 Feature selection (ablation)

Table 9. Cross-validated macro-F1 for different feature sets (training set)

Feature set	n	Logistic Regression	Random Forest
All 12 features	12	0.7956	0.9078
Without p1 (11 features)	11	0.7956	0.9105
Without p1-p4 (tau + g)	8	0.7931	0.9145
tau only	4	0.6945	0.7324
g only	4	0.6745	0.6520

p only	4	0.3895	0.4817

Removing p1 leaves Logistic Regression unchanged (its information is already contained in p2–p4) and slightly improves Random Forest (0.908 → 0.911), confirming that p1 is redundant — the UCI documentation itself describes p1 as non-predictive. Removing all four power variables also has almost no effect, and Random Forest even improves slightly (0.915), because fewer uninformative features compete for its random split candidates. Power alone carries no usable signal: Logistic Regression on p1–p4 predicts 'unstable' for everything (macro-F1 0.390 = baseline). Neither tau nor g alone is sufficient (macro-F1 0.65–0.73); good predictions require both groups together.

7.3 Feature engineering and hyperparameter tuning

Table 10. Top grid-search configurations (5-fold CV on training set). LR grid: degree ∈ {1, 2} × C ∈ {0.01, 0.1, 1, 10, 100}. RF grid (300 trees): max_depth ∈ {None, 12, 20} × min_samples_leaf ∈ {1, 3} × max_features ∈ {sqrt, 0.5}.

Model	Parameters	CV macro-F1	sd	Rank
Logistic Regression	{'C': 10, 'degree': 2}	0.9337	0.0074	1
Logistic Regression	{'C': 100, 'degree': 2}	0.9333	0.0069	2
Logistic Regression	{'C': 1, 'degree': 2}	0.9332	0.0073	3
Logistic Regression	{'C': 0.1, 'degree': 2}	0.9218	0.0049	4
Random Forest	{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 1}	0.9102	0.0102	1
Random Forest	{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1}	0.9099	0.0103	2
Random Forest	{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 3}	0.9077	0.0079	3
Random Forest	{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 3}	0.9077	0.0079	3

For Logistic Regression, the grid included PolynomialFeatures of degree 1 (the original model) and degree 2, which adds all squares and pairwise products such as tau1·g1 (90 features). Degree 2 with weak regularisation (C = 10) raised cross-validated macro-F1 from 0.796 to **0.934**, confirming the hypothesis in Section 7.1: the linear model was limited by its inputs, not by the algorithm. In the expanded feature space a linear boundary corresponds to a smooth quadratic boundary in the original space, which suits this physically smooth problem better than Random Forest's step-shaped boundary.

For Random Forest, limiting depth or leaf size and changing max_features altered cross-validated macro-F1 by less than 0.01 (best: max_depth = 20, max_features = sqrt, 0.910 vs 0.908 default). The default forest was already near its best; its remaining errors come from approximating a smooth boundary with axis-aligned splits, which tuning these parameters cannot remove.

7.4 Decision threshold and the cost of missed instabilities

By default an 'unstable' prediction requires P(unstable) > 0.5. Because a missed instability is more dangerous than a false alarm, we searched thresholds on out-of-fold training probabilities and selected,

for each tuned model, the highest threshold that still achieved at least 98% unstable recall (0.25 for Logistic Regression, 0.40 for Random Forest). Figure 9 shows the trade-off: lower thresholds increase recall but reduce precision.

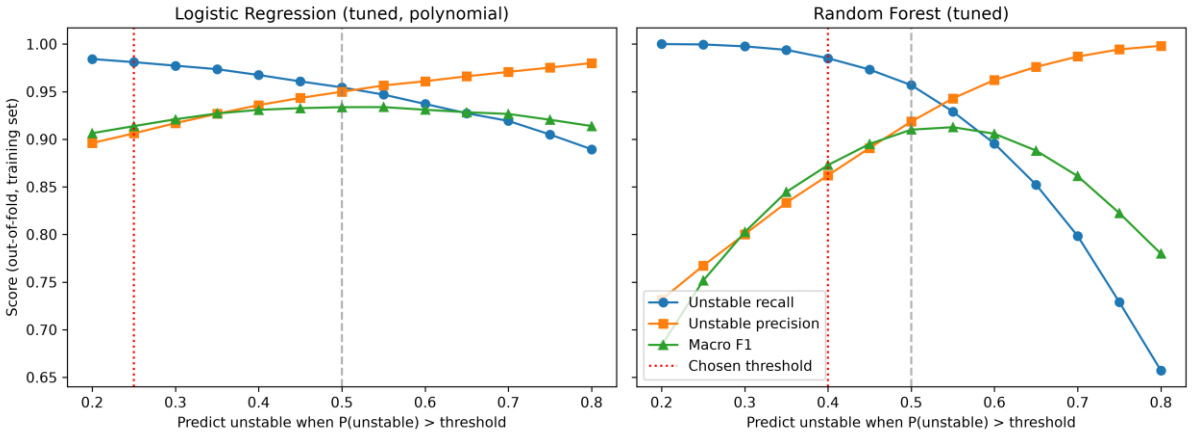


Figure 9. Unstable recall, unstable precision and macro-F1 versus decision threshold (out-of-fold training probabilities). Red line: chosen threshold.

7.5 Final comparison on the held-out test set

Table 11. Test-set comparison of all models (each model trained on the full training set; threshold chosen on training folds only)

Model	Thr.	Accuracy	Macro-F1	AUC	Unst. recall	Stab. recall	False stable	False unst.
Majority baseline	-	0.6380	0.3895	0.5000	1.0000	0.0000	0	724
LR	0.5	0.8200	0.8009	0.8920	0.8856	0.7044	146	214
RF	0.5	0.9290	0.9220	0.9823	0.9624	0.8702	48	94
LR (tuned, polynomial)	0.5	0.9555	0.9518	0.9897	0.9647	0.9392	45	44
RF (tuned)	0.5	0.9270	0.9201	0.9829	0.9569	0.8743	55	91
LR (tuned, polynomial), recall-oriented threshold	0.25	0.9405	0.9338	0.9897	0.9867	0.8591	17	102
RF (tuned), recall-oriented threshold	0.4	0.8980	0.8827	0.9829	0.9867	0.7417	17	187

Tuned Logistic Regression with polynomial features is the best model: 95.6% accuracy, macro-F1 0.952 and AUC 0.990, compared with 92.9% / 0.922 / 0.982 for the default Random Forest. It makes slightly fewer missed instabilities than Random Forest (45 vs 48) and less than half as many false alarms (44 vs 94) and, unlike Random Forest, recognises stable grids nearly as well as unstable ones (stable recall 93.9%). The cross-validation ranking predicted this result, so it is not a consequence of choosing models on the test set. The tuned Random Forest (92.7%) performed slightly below the default (92.9%); this difference of four test records is within the cross-validation variability, and it illustrates that a small gain in cross-validation does not guarantee a gain on new data.

With the recall-oriented thresholds, both tuned models miss only 17 of 1,276 unstable test grids (unstable recall 98.7%). The price differs: Logistic Regression raises 102 false alarms (accuracy 94.1%), whereas

Random Forest raises 187 (89.8%). Which operating point is preferable depends on the relative cost of an unhandled instability and an unnecessary intervention, which would have to be set by a grid operator, not by the classifier.

7.6 Summary of findings

- The operating parameters predict stability well: all models clearly beat the 63.8% baseline.
- With the original features, Random Forest outperforms Logistic Regression because stability depends on non-linear interactions between parameters.
- When interactions are provided explicitly, Logistic Regression becomes the most accurate model (95.6%): the choice of features mattered more than the choice of algorithm.
- Reaction times and price elasticities drive stability; power values carry almost no information and p1 is exactly redundant.
- Adjusting the decision threshold can cut missed instabilities by about 60% at a controlled cost in false alarms.

8. Limitations and Future Work

8.1 Limitations

- **Simulated data.** Labels come from a linearised mathematical model, not from measurements. The models learn to reproduce that model's stability rule; they say nothing about real grids with measurement noise, non-linear dynamics, faults or changing topology.
- **One fixed topology.** All records describe the same four-node star network with one producer. The models cannot be applied to larger or different networks without retraining on new data.
- **Idealised sampling.** Inputs are sampled independently and uniformly over fixed ranges; real operating points are correlated and concentrated in narrower regions, so performance on realistic distributions may differ.
- **Binary label.** Converting stab to stable/unstable discards how close a configuration is to the boundary; configurations with stab near zero are inherently hard to classify (Section 6.3).
- **Limited search.** Only two degree options and a small Random Forest grid were tested; the threshold rule (≥ 98% recall) is an illustrative choice rather than a cost-based optimum.
- **Not a safety tool.** Even 98.7% unstable recall leaves missed instabilities; the models are an academic demonstration, not a substitute for engineering stability analysis.

8.2 Future work

- Engineer features guided by the physics (e.g., tau·g products per node only, ratios such as g/tau) to obtain a smaller, more interpretable model with similar accuracy.
- Evaluate other classical algorithms that model smooth non-linear boundaries, such as Support Vector Machines with an RBF kernel or gradient-boosted trees, under the same protocol.
- Predict the continuous stab score with regression and derive the label from it, so that the model also reports a stability margin.
- Exploit the symmetry of the three consumers, either by augmenting the data with permuted consumer orderings or with symmetry-aware features, to test whether more data helps.

- Choose decision thresholds from explicit costs supplied by domain experts, and calibrate probabilities so that they can be read as risk estimates.
- Validate on data from larger topologies, noisy inputs or real measurements before drawing any operational conclusions.

9. Conclusion

This study showed that the stability of a simulated decentralised smart grid can be predicted accurately from its operating parameters once the leaking stab column is removed. With original features, Random Forest (92.9% accuracy) was markedly better than Logistic Regression (82.0%) because stability depends on interactions between reaction delays and price elasticities. Supplying those interactions as polynomial features made Logistic Regression the best model (95.6% accuracy, macro-F1 0.952), showing that understanding the structure of the problem improved accuracy more than tuning the algorithm. Reaction times and elasticities carry nearly all the signal, and threshold adjustment can reduce missed instabilities to 17 of 1,276. Because the data are simulated from a single idealised network, these results demonstrate that the stability rule is learnable, not that the models are ready for operational use.

10. Individual Contributions

[Both members must review and correct this table so that it reflects the work each person actually did. It is pre-filled from the GitHub commit history where that is known.]

Table 12. Contribution of each member

Task	Charuka Kavinda	Nadeera Rajapaksha
Dataset selection, source verification, audit	[Confirm]	[Confirm]
EDA and leakage analysis	[Confirm]	[Confirm]
Preprocessing pipeline and modular src/ package	Charuka Kavinda (commit bd7eafa)	
Logistic Regression	[Confirm]	[Confirm]
Random Forest	[Confirm]	[Confirm]
Evaluation and model comparison	Charuka Kavinda (commit bd7eafa)	
Interactive Streamlit application (app.py)		Nadeera Rajapaksha (commit d4efea1)
Improvement experiments (notebook 07)	[Confirm]	[Confirm]
Report writing	[Confirm sections]	[Confirm sections]

References

Arzamasov, V. (2018) Electrical Grid Stability Simulated Data [Dataset]. UCI Machine Learning Repository. Available at: https://doi.org/10.24432/C5PG66.

Arzamasov, V., Böhm, K. and Jochem, P. (2018) 'Towards concise models of grid stability', in 2018 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm). Aalborg: IEEE, pp. 1–6. doi:10.1109/SmartGridComm.2018.8587498.

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.

Cox, D.R. (1958) 'The regression analysis of binary sequences', Journal of the Royal Statistical Society: Series B, 20(2), pp. 215–242.

Das, A. (n.d.) Electrical Grid Stability and Control Simulation [Kaggle dataset mirror]. Available at: https://www.kaggle.com/datasets/avikdas567/electrical-grid-stability-and-control-simulation.

Hastie, T., Tibshirani, R. and Friedman, J. (2009) The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd edn. New York: Springer.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.

Schäfer, B., Grabow, C., Auer, S., Kurths, J., Witthaut, D. and Timme, M. (2016) 'Taming instabilities in power grid networks by decentralized control', The European Physical Journal Special Topics, 225(3), pp. 569–582.

Strobl, C., Boulesteix, A.-L., Zeileis, A. and Hothorn, T. (2007) 'Bias in random forest variable importance measures: illustrations, sources and a solution', BMC Bioinformatics, 8, 25.

Appendix A. Source Code

All source code is reproduced below as text. It is also available in the public GitHub repository. Run order: python -m src.run_pipeline, python -m src.experiments, python -m src.train_predictor, streamlit run app.py; notebooks 01–07 reproduce the analysis step by step.

A.1 src/__init__.py

```
"""
Modular package initialization for electrical grid stability ML pipeline.
"""
```

A.2 src/data_checks.py

```
"""
Data audit and sanity checks for Electrical Grid Stability dataset.
"""


from pathlib import Path
import pandas as pd


# Documented generation ranges of the simulation (Arzamasov et al., 2018).
EXPECTED_RANGES = {
    "tau": (0.5, 10.0),   # reaction times tau1-tau4 (s)
    "p_consumer": (-2.0, -0.5),  # consumed power p2-p4
    "g": (0.05, 1.0),     # price elasticity g1-g4
}



def audit_dataset(filepath: str | Path) -> dict:
    df = pd.read_csv(filepath)
    report = {
        "shape": df.shape,
        "columns": df.columns.tolist(),
        "dtypes": df.dtypes.to_dict(),
        "missing_values": df.isnull().sum().to_dict(),
        "duplicate_rows": int(df.duplicated().sum()),
        "class_counts": df["stabf"].value_counts().to_dict(),
        "class_percentages": (df["stabf"].value_counts(normalize=True) * 100).round(2).to_dict(),
    }
    return report



def validity_checks(df: pd.DataFrame) -> pd.DataFrame:
    """
    Checks every feature against its documented simulation range, the power-balance
    constraint (p1 + p2 + p3 + p4 = 0) and the stab -> stabf labelling rule.
    Returns one row per check with the number of violating records.
    """
    tau_cols = ["tau1", "tau2", "tau3", "tau4"]
    p_cons_cols = ["p2", "p3", "p4"]
    g_cols = ["g1", "g2", "g3", "g4"]


    lo, hi = EXPECTED_RANGES["tau"]
    tau_bad = int(((df[tau_cols] < lo) | (df[tau_cols] > hi)).any(axis=1).sum())
    lo, hi = EXPECTED_RANGES["p_consumer"]
    p_bad = int(((df[p_cons_cols] < lo) | (df[p_cons_cols] > hi)).any(axis=1).sum())
    lo, hi = EXPECTED_RANGES["g"]
    g_bad = int(((df[g_cols] < lo) | (df[g_cols] > hi)).any(axis=1).sum())


    power_sum = df[["p1", "p2", "p3", "p4"]].sum(axis=1)
    balance_bad = int((power_sum.abs() > 1e-9).sum())


    # stabf must be "stable" exactly when stab <= 0 (target-leakage evidence)
    rule_bad = int(((df["stab"] <= 0) != (df["stabf"] == "stable")).sum())
    label_bad = int((~df["stabf"].isin(["stable", "unstable"])).sum())
```

```
    return pd.DataFrame({
        "Check": [
            "tau1-tau4 within [0.5, 10]",
            "p2-p4 within [-2, -0.5]",
            "g1-g4 within [0.05, 1]",
            "p1 + p2 + p3 + p4 = 0 (|sum| <= 1e-9)",
            "stabf == 'stable' exactly when stab <= 0",
            "stabf only contains 'stable'/'unstable'",
        ],
        "Violating rows": [tau_bad, p_bad, g_bad, balance_bad, rule_bad, label_bad],
        "Max |p1+p2+p3+p4|": [None, None, None, float(power_sum.abs().max()), None, None],
    })



if __name__ == "__main__":
    data_path = Path(__file__).resolve().parent.parent / "data" / "raw" / "electrical_grid_stability.csv"
    res = audit_dataset(data_path)
    print("Dataset Shape:", res["shape"])
    print("Class Counts:", res["class_counts"])
    print("Class Percentages (%):", res["class_percentages"])
    print("Missing Values Total:", sum(res["missing_values"].values()))
    print("Duplicates:", res["duplicate_rows"])
    print("\nValidity checks:")
    print(validity_checks(pd.read_csv(data_path)).to_string(index=False))
```

A.3 src/preprocessing.py

```
"""
Leakage-safe preprocessing and stratified dataset splitting.
"""


from pathlib import Path
from typing import Tuple
import pandas as pd
from sklearn.model_selection import train_test_split


FEATURE_COLUMNS = [
    "tau1", "tau2", "tau3", "tau4",
    "p1", "p2", "p3", "p4",
    "g1", "g2", "g3", "g4"
]
TARGET_COLUMN = "stabf"



def load_and_split_data(
    filepath: str | Path,
    test_size: float = 0.20,
    random_state: int = 42
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]:
    """
    Loads dataset, isolates the 12 operating features, strictly discards 'stab'
    to prevent target leakage, and splits with stratification.
    """
    df = pd.read_csv(filepath)
    X = df[FEATURE_COLUMNS]
    y = df[TARGET_COLUMN]


    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=test_size,
        random_state=random_state,
        stratify=y
    )


    return X_train, X_test, y_train, y_test
```

```
if __name__ == "__main__":
    data_path = Path(__file__).resolve().parent.parent / "data" / "raw" / "electrical_grid_stability.csv"
    X_tr, X_te, y_tr, y_te = load_and_split_data(data_path)
    print("Train Shape:", X_tr.shape, y_tr.shape)
    print("Test Shape:", X_te.shape, y_te.shape)
    print("Train Target Distribution:\n", y_tr.value_counts(normalize=True))
    print("Test Target Distribution:\n", y_te.value_counts(normalize=True))
```

A.4 src/train_models.py

```
"""
Model definition and training pipelines for Logistic Regression and Random Forest.
"""


from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier



def get_logistic_regression_pipeline(max_iter: int = 2000, random_state: int = 42) -> Pipeline:
    """
    StandardScaler inside a Pipeline guarantees feature normalization is fitted
    strictly on training folds without test-set leakage.
    """
    return Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=max_iter, random_state=random_state))
    ])



def get_random_forest_model(n_estimators: int = 200, random_state: int = 42, n_jobs: int = -1) ->
RandomForestClassifier:
    """
    Ensemble decision tree model capable of learning non-linear feature interactions.
    """
    return RandomForestClassifier(
        n_estimators=n_estimators,
        random_state=random_state,
        n_jobs=n_jobs
    )
```

A.5 src/evaluation.py

```
"""
Evaluation and metrics computation module.
"""


import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)



def evaluate_predictions(y_true, y_pred, labels=("stable", "unstable")) -> dict:
    acc = accuracy_score(y_true, y_pred)
    macro_f1 = f1_score(y_true, y_pred, average="macro")
    weighted_f1 = f1_score(y_true, y_pred, average="weighted")


    prec_stable = precision_score(y_true, y_pred, pos_label="stable", zero_division=0)
```

```
    rec_stable = recall_score(y_true, y_pred, pos_label="stable", zero_division=0)
    f1_stable = f1_score(y_true, y_pred, pos_label="stable", zero_division=0)

    prec_unstable = precision_score(y_true, y_pred, pos_label="unstable", zero_division=0)
    rec_unstable = recall_score(y_true, y_pred, pos_label="unstable", zero_division=0)
    f1_unstable = f1_score(y_true, y_pred, pos_label="unstable", zero_division=0)

    cm = confusion_matrix(y_true, y_pred, labels=list(labels))

    return {
        "accuracy": acc,
        "macro_f1": macro_f1,
        "weighted_f1": weighted_f1,
        "stable_precision": prec_stable,
        "stable_recall": rec_stable,
        "stable_f1": f1_stable,
        "unstable_precision": prec_unstable,
        "unstable_recall": rec_unstable,
        "unstable_f1": f1_unstable,
        "confusion_matrix": cm,
        "classification_report": classification_report(y_true, y_pred, digits=4, zero_division=0)
    }
```

A.6 src/run_pipeline.py

```
"""
Automated pipeline execution to train both models, evaluate on held-out test data,
and generate all output figures and metrics tables.
"""

from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.metrics import ConfusionMatrixDisplay

from src.preprocessing import load_and_split_data
from src.train_models import get_logistic_regression_pipeline, get_random_forest_model
from src.evaluation import evaluate_predictions



def run_full_pipeline():
    project_root = Path(__file__).resolve().parent.parent
    data_path = project_root / "data" / "raw" / "electrical_grid_stability.csv"
    figures_dir = project_root / "outputs" / "figures"
    metrics_dir = project_root / "outputs" / "metrics"

    figures_dir.mkdir(parents=True, exist_ok=True)
    metrics_dir.mkdir(parents=True, exist_ok=True)

    print("1. Loading and splitting dataset...")
    X_train, X_test, y_train, y_test = load_and_split_data(data_path)

    # Raw dataframe for EDA plots
    df_raw = pd.read_csv(data_path)

    print("2. Generating EDA figures...")
    # Plot 1: Target distribution
    plt.figure(figsize=(6, 4))
    sns.countplot(data=df_raw, x="stabf", hue="stabf", order=["unstable", "stable"],
                  palette={"unstable": "#e74c3c", "stable": "#2ecc71"}, legend=False)
    plt.title("Distribution of Stability Class (stabf)")
    plt.xlabel("Grid Stability Label")
    plt.ylabel("Sample Count")
    plt.tight_layout()
    plt.savefig(figures_dir / "target_distribution.png", dpi=300)
    plt.close()
```

```
# Plot 2: Boxplot of stab by stabf (Leakage illustration)
plt.figure(figsize=(6, 4))
sns.boxplot(data=df_raw, x="stabf", y="stab", hue="stabf", order=["unstable", "stable"],
            palette={"unstable": "#e74c3c", "stable": "#2ecc71"}, legend=False)
plt.axhline(0, color="black", linestyle="--", alpha=0.7, label="Threshold (stab=0)")
plt.title("Continuous Stability Score (stab) vs Target Class (stabf)")
plt.xlabel("Grid Stability Label")
plt.ylabel("Eigenvalue Stability Indicator (stab)")
plt.legend()
plt.tight_layout()
plt.savefig(figures_dir / "stab_vs_stabf_leakage.png", dpi=300)
plt.close()


# Plot 3: Feature Correlation Heatmap (12 operating features)
feature_cols = [c for c in df_raw.columns if c not in ["stab", "stabf"]]
plt.figure(figsize=(10, 8))
sns.heatmap(df_raw[feature_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Correlation Heatmap of 12 Operating Parameters")
plt.tight_layout()
plt.savefig(figures_dir / "feature_correlation_heatmap.png", dpi=300)
plt.close()


# Plot 4: Distribution of each operating feature by stability class
long_df = df_raw.melt(id_vars="stabf", value_vars=feature_cols, var_name="feature")
grid = sns.FacetGrid(long_df, col="feature", col_wrap=4, hue="stabf", sharex=False, sharey=False,
                     palette={"unstable": "#e74c3c", "stable": "#2ecc71"}, height=2.2, aspect=1.4)
grid.map(sns.kdeplot, "value", fill=True, alpha=0.35)
grid.add_legend(title="stabf")
grid.figure.suptitle("Operating Feature Distributions by Stability Class", y=1.02)
grid.savefig(figures_dir / "feature_distributions_by_class.png", dpi=300, bbox_inches="tight")
plt.close(grid.figure)


print("3. Training Logistic Regression pipeline...")
lr_pipe = get_logistic_regression_pipeline()
lr_pipe.fit(X_train, y_train)
y_pred_lr = lr_pipe.predict(X_test)


print("4. Training Random Forest classifier...")
rf_model = get_random_forest_model()
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)


print("5. Evaluating predictions...")
res_lr = evaluate_predictions(y_test, y_pred_lr)
res_rf = evaluate_predictions(y_test, y_pred_rf)


# Zero-rule baseline: always predict the majority class seen in training
majority_class = y_train.mode()[0]
baseline_acc = (y_test == majority_class).mean()


# Comparison Table
comparison_df = pd.DataFrame({
    "Metric": [
        "Baseline Zero-Rule",
        "Accuracy",
        "Macro F1-Score",
        "Weighted F1-Score",
        "Stable Precision",
        "Stable Recall",
        "Stable F1-Score",
        "Unstable Precision",
        "Unstable Recall",
        "Unstable F1-Score"
    ],
    "Logistic Regression": [
        round(baseline_acc, 4),
        round(res_lr["accuracy"], 4),
        round(res_lr["macro_f1"], 4),
        round(res_lr["weighted_f1"], 4),
        round(res_lr["stable_precision"], 4),
```

```
            round(res_lr["stable_recall"], 4),
            round(res_lr["stable_f1"], 4),
            round(res_lr["unstable_precision"], 4),
            round(res_lr["unstable_recall"], 4),
            round(res_lr["unstable_f1"], 4),
        ],
        "Random Forest": [
            round(baseline_acc, 4),
            round(res_rf["accuracy"], 4),
            round(res_rf["macro_f1"], 4),
            round(res_rf["weighted_f1"], 4),
            round(res_rf["stable_precision"], 4),
            round(res_rf["stable_recall"], 4),
            round(res_rf["stable_f1"], 4),
            round(res_rf["unstable_precision"], 4),
            round(res_rf["unstable_recall"], 4),
            round(res_rf["unstable_f1"], 4),
        ]
    })
    comparison_df.to_csv(metrics_dir / "model_comparison.csv", index=False)
    print("\n=== Model Comparison Table ===")
    print(comparison_df.to_string(index=False))


    # Confusion Matrices Figure
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    ConfusionMatrixDisplay(res_lr["confusion_matrix"], display_labels=["stable", "unstable"]).plot(
        ax=axes[0], cmap="Blues", values_format="d"
    )
    axes[0].set_title(f"Logistic Regression\nAccuracy: {res_lr['accuracy']:.4f}")


    ConfusionMatrixDisplay(res_rf["confusion_matrix"], display_labels=["stable", "unstable"]).plot(
        ax=axes[1], cmap="Greens", values_format="d"
    )
    axes[1].set_title(f"Random Forest\nAccuracy: {res_rf['accuracy']:.4f}")
    plt.tight_layout()
    plt.savefig(figures_dir / "confusion_matrices.png", dpi=300)
    plt.close()


    # Feature Importance Figure (Random Forest)
    importances = pd.Series(rf_model.feature_importances_, index=feature_cols).sort_values(ascending=False)
    plt.figure(figsize=(10, 5))
    sns.barplot(x=importances.values, y=importances.index, hue=importances.index, palette="viridis", legend=False)
    plt.title("Random Forest Feature Importances (MDI)")
    plt.xlabel("Mean Decrease in Impurity")
    plt.ylabel("Grid Parameter")
    plt.tight_layout()
    plt.savefig(figures_dir / "rf_feature_importance.png", dpi=300)
    plt.close()


    print("\nPipeline execution complete! Figures and metrics saved successfully.")



if __name__ == "__main__":
    run_full_pipeline()
```

A.7 src/experiments.py

```
"""
Validation and accuracy-improvement experiments.

Every model-selection decision here is made with stratified 5-fold cross-validation
on the TRAINING set only. The held-out test set is used once, at the end, to report
the final scores of the baseline and improved models.
"""


from pathlib import Path


import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd
import seaborn as sns
from sklearn.base import clone
from sklearn.decomposition import PCA
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score, recall_score, roc_auc_score, roc_curve
)
from sklearn.model_selection import (
    GridSearchCV, StratifiedKFold, cross_val_predict, cross_validate
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler


from src.preprocessing import FEATURE_COLUMNS, load_and_split_data
from src.train_models import get_logistic_regression_pipeline, get_random_forest_model


RANDOM_STATE = 42
CV = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
SCORING = {"accuracy": "accuracy", "macro_f1": "f1_macro", "roc_auc": "roc_auc"}


FEATURE_GROUPS = {
    "All 12 features": FEATURE_COLUMNS,
    "Without p1 (11 features)": [c for c in FEATURE_COLUMNS if c != "p1"],
    "Without p1-p4 (tau + g)": [c for c in FEATURE_COLUMNS if not c.startswith("p")],
    "tau only": ["tau1", "tau2", "tau3", "tau4"],
    "g only": ["g1", "g2", "g3", "g4"],
    "p only": ["p1", "p2", "p3", "p4"],
}



def get_polynomial_logistic_pipeline(degree: int = 2, C: float = 1.0) -> Pipeline:
    """
    Logistic Regression on degree-2 polynomial features (squares and pairwise
    interactions such as tau1*g1), giving the linear model a non-linear boundary.
    """
    return Pipeline([
        ("poly", PolynomialFeatures(degree=degree, include_bias=False)),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(C=C, max_iter=5000, random_state=RANDOM_STATE)),
    ])



def cv_summary(model, X, y) -> dict:
    """Mean and standard deviation of 5-fold CV scores plus mean train-fold accuracy."""
    res = cross_validate(model, X, y, cv=CV, scoring=SCORING, return_train_score=True, n_jobs=-1)
    out = {}
    for name in SCORING:
        out[f"cv_{name}_mean"] = res[f"test_{name}"].mean()
        out[f"cv_{name}_std"] = res[f"test_{name}"].std()
    out["cv_train_accuracy_mean"] = res["train_accuracy"].mean()
    return out



def test_scores(model, X_test, y_test, threshold: float = 0.5) -> dict:
    """Test-set metrics. `threshold`: predict "unstable" when P(unstable) > threshold (0.5 matches
model.predict)."""
    p_unstable = model.predict_proba(X_test)[:, list(model.classes_).index("unstable")]
    y_pred = np.where(p_unstable > threshold, "unstable", "stable")
    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "macro_f1": f1_score(y_test, y_pred, average="macro"),
        "roc_auc": roc_auc_score(y_test == "unstable", p_unstable),
        "unstable_precision": precision_score(y_test, y_pred, pos_label="unstable"),
        "unstable_recall": recall_score(y_test, y_pred, pos_label="unstable"),
        "stable_recall": recall_score(y_test, y_pred, pos_label="stable"),
        "false_stable": int(((y_test == "unstable") & (y_pred == "stable")).sum()),
        "false_unstable": int(((y_test == "stable") & (y_pred == "unstable")).sum()),
    }
```

```
def run_experiments():
    project_root = Path(__file__).resolve().parent.parent
    data_path = project_root / "data" / "raw" / "electrical_grid_stability.csv"
    figures_dir = project_root / "outputs" / "figures"
    metrics_dir = project_root / "outputs" / "metrics"
    figures_dir.mkdir(parents=True, exist_ok=True)
    metrics_dir.mkdir(parents=True, exist_ok=True)


    X_train, X_test, y_train, y_test = load_and_split_data(data_path)
    baseline_models = {
        "Logistic Regression": get_logistic_regression_pipeline(),
        "Random Forest": get_random_forest_model(),
    }


    # ------------------------------------------------------------------
    # 1. Cross-validation and overfitting check for the baseline models
    # ------------------------------------------------------------------
    print("1. Cross-validating baseline models...")
    rows = []
    for name, model in baseline_models.items():
        row = {"Model": name, **cv_summary(model, X_train, y_train)}
        fitted = clone(model).fit(X_train, y_train)
        row["full_train_accuracy"] = accuracy_score(y_train, fitted.predict(X_train))
        row["test_accuracy"] = accuracy_score(y_test, fitted.predict(X_test))
        rows.append(row)
    cv_df = pd.DataFrame(rows).round(4)
    cv_df.to_csv(metrics_dir / "cv_and_overfitting.csv", index=False)
    print(cv_df.to_string(index=False))


    # ------------------------------------------------------------------
    # 2. Feature-group ablation (redundant p1, feature families)
    # ------------------------------------------------------------------
    print("\n2. Feature-group ablation (CV on training set)...")
    rows = []
    for group, cols in FEATURE_GROUPS.items():
        for name, model in baseline_models.items():
            s = cv_summary(model, X_train[cols], y_train)
            rows.append({
                "Feature set": group, "Model": name, "n_features": len(cols),
                "cv_accuracy": s["cv_accuracy_mean"], "cv_macro_f1": s["cv_macro_f1_mean"],
                "cv_macro_f1_std": s["cv_macro_f1_std"],
            })
    ablation_df = pd.DataFrame(rows).round(4)
    ablation_df.to_csv(metrics_dir / "feature_ablation.csv", index=False)
    print(ablation_df.to_string(index=False))


    # ------------------------------------------------------------------
    # 3. Dimensionality reduction: PCA on standardised training features
    # ------------------------------------------------------------------
    print("\n3. PCA explained variance...")
    pca = PCA().fit(StandardScaler().fit_transform(X_train))
    pca_df = pd.DataFrame({
        "component": np.arange(1, len(FEATURE_COLUMNS) + 1),
        "explained_variance_ratio": pca.explained_variance_ratio_,
        "cumulative": np.cumsum(pca.explained_variance_ratio_),
    }).round(6)
    pca_df.to_csv(metrics_dir / "pca_explained_variance.csv", index=False)
    print(pca_df.to_string(index=False))


    fig, ax = plt.subplots(figsize=(7, 4))
    ax.bar(pca_df["component"], pca_df["explained_variance_ratio"], color="#4c72b0", label="Per component")
    ax.plot(pca_df["component"], pca_df["cumulative"], "o-", color="#dd8452", label="Cumulative")
    ax.set_xticks(pca_df["component"])
    ax.set_xlabel("Principal component")
    ax.set_ylabel("Explained variance ratio")
    ax.set_title("PCA on 12 Standardised Operating Features (training set)")
    ax.legend()
    fig.tight_layout()
    fig.savefig(figures_dir / "pca_explained_variance.png", dpi=300)
    plt.close(fig)
```

```
# ------------------------------------------------------------
# 4. Hyperparameter tuning (GridSearchCV on training set only)
# ------------------------------------------------------------
print("\n4. Tuning Logistic Regression (with/without polynomial features)...")
lr_grid = GridSearchCV(
    Pipeline([
        ("poly", PolynomialFeatures(include_bias=False)),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=5000, random_state=RANDOM_STATE)),
    ]),
    param_grid={"poly__degree": [1, 2], "classifier__C": [0.01, 0.1, 1, 10, 100]},
    scoring="f1_macro", cv=CV, n_jobs=-1,
).fit(X_train, y_train)


print("   Tuning Random Forest...")
rf_grid = GridSearchCV(
    get_random_forest_model(n_estimators=300),
    param_grid={
        "max_depth": [None, 12, 20],
        "min_samples_leaf": [1, 3],
        "max_features": ["sqrt", 0.5],
    },
    scoring="f1_macro", cv=CV, n_jobs=-1,
).fit(X_train, y_train)


tuning_rows = []
for name, grid in [("Logistic Regression", lr_grid), ("Random Forest", rf_grid)]:
    res = pd.DataFrame(grid.cv_results_)
    for _, r in res.iterrows():
        tuning_rows.append({
            "Model": name,
            "params": str(r["params"]),
            "cv_macro_f1_mean": r["mean_test_score"],
            "cv_macro_f1_std": r["std_test_score"],
            "rank": int(r["rank_test_score"]),
        })
    print(f"   Best {name}: {grid.best_params_}  CV macro-F1 = {grid.best_score_:.4f}")
tuning_df = pd.DataFrame(tuning_rows).sort_values(["Model", "rank"]).round(4)
tuning_df.to_csv(metrics_dir / "hyperparameter_tuning.csv", index=False)


tuned_models = {
    "Logistic Regression (tuned, polynomial)": lr_grid.best_estimator_,
    "Random Forest (tuned)": rf_grid.best_estimator_,
}


# ------------------------------------------------------------
# 5. Decision-threshold analysis for missed instabilities
#    Threshold chosen on out-of-fold training probabilities, never on test.
# ------------------------------------------------------------
print("\n5. Decision-threshold analysis (out-of-fold training probabilities)...")
thresholds = np.round(np.arange(0.20, 0.81, 0.05), 2)
rows = []
chosen_thresholds = {}
for name, model in tuned_models.items():
    oof = cross_val_predict(clone(model), X_train, y_train, cv=CV, method="predict_proba", n_jobs=-1)
    p_unstable = oof[:, list(model.classes_).index("unstable")]
    for t in thresholds:
        y_pred = np.where(p_unstable > t, "unstable", "stable")
        rows.append({
            "Model": name, "threshold": t,
            "unstable_recall": recall_score(y_train, y_pred, pos_label="unstable"),
            "unstable_precision": precision_score(y_train, y_pred, pos_label="unstable"),
            "macro_f1": f1_score(y_train, y_pred, average="macro"),
            "accuracy": accuracy_score(y_train, y_pred),
        })
    # Highest threshold that still reaches >= 98% unstable recall on the training folds
    model_rows = [r for r in rows if r["Model"] == name and r["unstable_recall"] >= 0.98]
    chosen_thresholds[name] = max(r["threshold"] for r in model_rows) if model_rows else 0.5
    print(f"   {name}: threshold for >=98% unstable recall = {chosen_thresholds[name]}")
threshold_df = pd.DataFrame(rows).round(4)
threshold_df.to_csv(metrics_dir / "threshold_analysis.csv", index=False)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)
for ax, name in zip(axes, tuned_models):
    d = threshold_df[threshold_df["Model"] == name]
    ax.plot(d["threshold"], d["unstable_recall"], "o-", label="Unstable recall")
    ax.plot(d["threshold"], d["unstable_precision"], "s-", label="Unstable precision")
    ax.plot(d["threshold"], d["macro_f1"], "^-", label="Macro F1")
    ax.axvline(0.5, color="grey", ls="--", alpha=0.6)
    ax.axvline(chosen_thresholds[name], color="red", ls=":", label="Chosen threshold")
    ax.set_title(name)
    ax.set_xlabel("Predict unstable when P(unstable) > threshold")
axes[0].set_ylabel("Score (out-of-fold, training set)")
axes[1].legend(loc="lower left")
fig.tight_layout()
fig.savefig(figures_dir / "threshold_tradeoff.png", dpi=300)
plt.close(fig)


# ---------------------------------------------------------------
# 6. Final held-out test evaluation (used once)
# ---------------------------------------------------------------
print("\n6. Final held-out test evaluation...")
final_models = {name: clone(m).fit(X_train, y_train) for name, m in baseline_models.items()}
final_models.update(tuned_models)  # GridSearchCV already refit these on the full training set


rows = [{
    "Model": "Majority baseline (always unstable)", "threshold": "-",
    "accuracy": (y_test == "unstable").mean(), "macro_f1": f1_score(
        y_test, np.full(len(y_test), "unstable"), average="macro"),
    "roc_auc": 0.5, "unstable_precision": (y_test == "unstable").mean(),
    "unstable_recall": 1.0, "stable_recall": 0.0,
    "false_stable": 0, "false_unstable": int((y_test == "stable").sum()),
}]
for name, model in final_models.items():
    rows.append({"Model": name, "threshold": 0.5, **test_scores(model, X_test, y_test)})
for name, model in tuned_models.items():
    t = chosen_thresholds[name]
    rows.append({"Model": f"{name}, recall-oriented threshold", "threshold": t,
                 **test_scores(model, X_test, y_test, threshold=t)})
final_df = pd.DataFrame(rows)
num_cols = final_df.select_dtypes("number").columns
final_df[num_cols] = final_df[num_cols].round(4)
final_df.to_csv(metrics_dir / "final_model_comparison.csv", index=False)
print(final_df.to_string(index=False))


# ROC curves on the test set
fig, ax = plt.subplots(figsize=(6.5, 5.5))
for name, model in final_models.items():
    p = model.predict_proba(X_test)[:, list(model.classes_).index("unstable")]
    fpr, tpr, _ = roc_curve(y_test == "unstable", p)
    ax.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc_score(y_test == 'unstable', p):.3f})")
ax.plot([0, 1], [0, 1], "k--", alpha=0.5, label="Chance")
ax.set_xlabel("False positive rate (stable predicted unstable)")
ax.set_ylabel("True positive rate (unstable recall)")
ax.set_title("ROC Curves on Held-Out Test Set (positive = unstable)")
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout()
fig.savefig(figures_dir / "roc_curves.png", dpi=300)
plt.close(fig)


# ---------------------------------------------------------------
# 7. Permutation importance (less biased than impurity-based MDI)
# ---------------------------------------------------------------
print("\n7. Permutation importance on test set...")
rows = []
for name in ["Logistic Regression", "Random Forest"]:
    pi = permutation_importance(final_models[name], X_test, y_test, scoring="f1_macro",
                                n_repeats=10, random_state=RANDOM_STATE, n_jobs=-1)
    for f, m, s in zip(FEATURE_COLUMNS, pi.importances_mean, pi.importances_std):
        rows.append({"Model": name, "feature": f, "importance_mean": m, "importance_std": s})
perm_df = pd.DataFrame(rows).round(4)
perm_df.to_csv(metrics_dir / "permutation_importance.csv", index=False)
```

```
    fig, ax = plt.subplots(figsize=(9, 5))
    order = (perm_df[perm_df["Model"] == "Random Forest"]
             .sort_values("importance_mean", ascending=False)["feature"])
    sns.barplot(data=perm_df, x="importance_mean", y="feature", hue="Model", order=order, ax=ax)
    ax.set_xlabel("Mean drop in macro-F1 when feature is shuffled")
    ax.set_ylabel("Grid parameter")
    ax.set_title("Permutation Feature Importance (held-out test set)")
    fig.tight_layout()
    fig.savefig(figures_dir / "permutation_importance.png", dpi=300)
    plt.close(fig)

    print("\nExperiments complete. Results saved to outputs/metrics and outputs/figures.")
    return {
        "cv": cv_df, "ablation": ablation_df, "pca": pca_df, "tuning": tuning_df,
        "thresholds": threshold_df, "final": final_df, "permutation": perm_df,
        "best_params": {"Logistic Regression": lr_grid.best_params_, "Random Forest": rf_grid.best_params_},
        "chosen_thresholds": chosen_thresholds,
    }



if __name__ == "__main__":
    run_experiments()
```

A.8 src/train_predictor.py

```
"""Train and save the Random Forest model used by the interactive app."""

from pathlib import Path

import joblib

from src.preprocessing import load_and_split_data
from src.train_models import get_random_forest_model

PROJECT_ROOT = Path(__file__).resolve().parent.parent
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"



def train_model():
    """
    Fits the Random Forest on the same 80% training split used in the report, so the
    20% test split stays unseen and the app can show honest held-out predictions.
    """
    X_train, _, y_train, _ = load_and_split_data(DATA_PATH)

    model = get_random_forest_model()
    model.fit(X_train, y_train)
    return model, len(X_train)



def train_and_save_model() -> Path:
    model_path = PROJECT_ROOT / "outputs" / "models" / "random_forest.joblib"
    model, row_count = train_model()

    model_path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, model_path)

    print(f"Trained Random Forest on {row_count:,} training rows (test split held out).")
    print(f"Saved model to: {model_path}")
    return model_path



if __name__ == "__main__":
    train_and_save_model()
```

A.9 app.py

```
"""
Smart Electrical Grid Stability AI Platform
Interactive Streamlit Diagnostics, Simulation, and Decision Support System
"""


from pathlib import Path
import joblib
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st


from src.preprocessing import FEATURE_COLUMNS, TARGET_COLUMN, load_and_split_data
from src.train_predictor import train_model


# ------------------------------------------------------------
# Page Configuration
# ------------------------------------------------------------
st.set_page_config(
    page_title="Smart Grid Stability AI Platform",
    page_icon=" ",
    layout="wide",
    initial_sidebar_state="expanded"
)


PROJECT_ROOT = Path(__file__).resolve().parent
MODEL_PATH = PROJECT_ROOT / "outputs" / "models" / "random_forest.joblib"
METRICS_PATH = PROJECT_ROOT / "outputs" / "metrics" / "model_comparison.csv"
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


# ------------------------------------------------------------
# Custom Styling & Glassmorphic Design System
# ------------------------------------------------------------
st.markdown("""
<style>
    @import
url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&family=JetBrains+Mono:wght@400;60
0&display=swap');


    html, body, [class*="css"] {
        font-family: 'Outfit', sans-serif;
    }


    code, .stCode {
        font-family: 'JetBrains Mono', monospace !important;
    }


    /* ------------------------------------------------------------
       Streamlit Header & 3-Dots Menu Customization
       ------------------------------------------------------------ */
    /* 1. Hide Deploy Button */
    .stAppDeployButton,
    [data-testid="stAppDeployButton"],
    .stDeployButton,
    [data-testid="stDeployButton"],
    header [data-testid="stToolbarActions"],
    header button:has([data-testid="stDeployButton"]) {
        display: none !important;
        visibility: hidden !important;
        width: 0 !important;
        height: 0 !important;
        padding: 0 !important;
        margin: 0 !important;
        overflow: hidden !important;
    }


    /* 2. Theme Switcher: Ensure container is visible */
    [data-testid="stThemeSwitcher"] {
```

```
    display: flex !important;
    width: 100% !important;
    gap: 8px !important;
}

/* Explicitly keep System and Dark buttons visible (each takes 50%) */
[data-testid="stMainMenuItem-theme-System"],
[data-testid="stMainMenuItem-theme-Dark"],
[data-testid="stThemeSwitcher"] > *:first-child,
[data-testid="stThemeSwitcher"] > *:last-child {
    display: flex !important;
    visibility: visible !important;
    flex: 1 1 50% !important;
}

/* Hide 'Light' mode button */
[data-testid="stMainMenuItem-theme-Light"],
[data-testid="stThemeSwitcher"] > *:nth-child(2) {
    display: none !important;
    visibility: hidden !important;
}

/* 3. Hide all menu items & dividers below the theme switcher */
[data-testid="stThemeSwitcher"] ~ *,
[data-testid="stMainMenuDivider"],
[data-testid="stMainMenuItem-rerun"],
[data-testid="stMainMenuItem-autoRerun"],
[data-testid="stMainMenuItem-clearCache"],
[data-testid="stMainMenuItem-print"],
[data-testid="stMainMenuItem-recordScreencast"],
[data-testid="stMainMenuItem-about"],
[data-testid="stMainMenuList"] ~ *,
.stMenuVersionCopyButton {
    display: none !important;
}

/* Clean popover padding */
[data-testid="stMainMenuList"] {
    padding-bottom: 4px !important;
    margin-bottom: 0 !important;
}

/* Sidebar Brand Title (Adaptive Light/Dark/System Theme) */
.brand-title {
    margin: 0 !important;
    font-size: 1.25rem !important;
    font-weight: 800 !important;
    color: var(--text-color, #0f172a) !important;
    letter-spacing: -0.01em !important;
    line-height: 1.2 !important;
}

.brand-subtitle {
    font-size: 0.72rem !important;
    color: #0284c7 !important;
    font-weight: 700 !important;
    letter-spacing: 0.06em !important;
    text-transform: uppercase !important;
}

@media (prefers-color-scheme: dark) {
    .brand-subtitle {
        color: #38bdf8 !important;
    }
}

[data-theme="dark"] .brand-subtitle,
[data-base="dark"] .brand-subtitle {
    color: #38bdf8 !important;
}
```

```
/* ------------------------------------------------------
   Clean Selectbox & Dropdown Styling
   ------------------------------------------------------ */
/* Selectbox Input Container */
.stSelectbox div[data-baseweb="select"] > div,
div[data-testid="stSelectbox"] div:has(> input) {
    border-radius: 10px !important;
}

/* Selectbox Input Text */
.stSelectbox input,
div[data-testid="stSelectbox"] input {
    font-size: 0.9rem !important;
    line-height: 1.3 !important;
}

/* Ensure Virtual Dropdown Popover floats cleanly above all sidebar elements */
div[data-testid="stSelectboxVirtualDropdown"] {
    z-index: 999999 !important;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.5) !important;
    border-radius: 10px !important;
}

/* Main Container Padding */
.block-container {
    padding-top: 1.8rem;
    padding-bottom: 3rem;
    max-width: 1300px;
}

/* Hero Banner */
.hero-container {
    background: linear-gradient(135deg, rgba(15, 23, 42, 0.95) 0%, rgba(30, 41, 59, 0.9) 100%);
    border: 1px solid rgba(56, 189, 248, 0.25);
    border-radius: 20px;
    padding: 26px 32px;
    margin-bottom: 24px;
    box-shadow: 0 12px 36px -8px rgba(0, 0, 0, 0.5), 0 0 20px rgba(56, 189, 248, 0.1);
    backdrop-filter: blur(12px);
}

.hero-badge {
    display: inline-flex;
    align-items: center;
    gap: 6px;
    background: rgba(56, 189, 248, 0.15);
    color: #38bdf8;
    padding: 4px 14px;
    border-radius: 9999px;
    font-size: 0.8rem;
    font-weight: 600;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    border: 1px solid rgba(56, 189, 248, 0.3);
    margin-bottom: 12px;
}

.hero-title {
    font-size: 2.2rem;
    font-weight: 800;
    letter-spacing: -0.02em;
    background: linear-gradient(90deg, #f8fafc 0%, #38bdf8 50%, #818cf8 100%);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    margin: 0 0 8px 0;
}

.hero-subtitle {
    color: #94a3b8;
    font-size: 1.05rem;
    font-weight: 400;
    line-height: 1.5;
```

```
    margin: 0;
}

/* Metric Cards Grid */
.kpi-card {
    background: rgba(30, 41, 59, 0.65);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: 14px;
    padding: 16px 20px;
    transition: transform 0.2s ease, border-color 0.2s ease;
}
.kpi-card:hover {
    transform: translateY(-2px);
    border-color: rgba(56, 189, 248, 0.4);
}
.kpi-label {
    color: #64748b;
    font-size: 0.78rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-bottom: 4px;
}
.kpi-value {
    font-size: 1.6rem;
    font-weight: 700;
    color: #f1f5f9;
    font-family: 'JetBrains Mono', monospace;
}
.kpi-caption {
    font-size: 0.75rem;
    color: #10b981;
    font-weight: 500;
    margin-top: 2px;
}

/* Section Cards */
.glass-card {
    background: rgba(30, 41, 59, 0.5);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: 16px;
    padding: 22px;
    margin-bottom: 20px;
    backdrop-filter: blur(8px);
}
.glass-card-header {
    display: flex;
    align-items: center;
    gap: 10px;
    font-size: 1.15rem;
    font-weight: 700;
    color: #f8fafc;
    margin-bottom: 14px;
    padding-bottom: 8px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.08);
}

/* Verdict Card */
.verdict-box {
    border-radius: 16px;
    padding: 24px;
    text-align: center;
    margin-bottom: 20px;
    box-shadow: 0 10px 30px -10px rgba(0, 0, 0, 0.6);
    animation: fadeIn 0.4s ease-in-out;
}
.verdict-stable {
    background: linear-gradient(135deg, rgba(6, 78, 59, 0.9) 0%, rgba(6, 95, 70, 0.7) 100%);
    border: 2px solid #10b981;
    box-shadow: 0 0 25px rgba(16, 185, 129, 0.25);
}
.verdict-unstable {
    background: linear-gradient(135deg, rgba(127, 29, 29, 0.9) 0%, rgba(153, 27, 27, 0.7) 100%);
    border: 2px solid #ef4444;
```

```
        box-shadow: 0 0 25px rgba(239, 68, 68, 0.25);
    }

    .verdict-tag {
        font-size: 0.85rem;
        font-weight: 700;
        letter-spacing: 0.1em;
        text-transform: uppercase;
        margin-bottom: 6px;
    }
    .verdict-title {
        font-size: 2.2rem;
        font-weight: 800;
        letter-spacing: -0.01em;
        margin: 0;
        color: #ffffff;
    }

    /* Custom Streamlit Tabs */
    .stTabs [data-baseweb="tab-list"] {
        gap: 12px;
        border-bottom: 1px solid rgba(255, 255, 255, 0.1);
        padding-bottom: 4px;
    }
    .stTabs [data-baseweb="tab"] {
        border-radius: 10px 10px 0 0;
        padding: 10px 20px;
        font-weight: 600;
        font-size: 0.95rem;
        background: transparent;
        color: #94a3b8;
        border: none;
    }
    .stTabs [aria-selected="true"] {
        background: rgba(56, 189, 248, 0.12) !important;
        color: #38bdf8 !important;
        border-bottom: 2px solid #38bdf8 !important;
    }

    /* Badges */
    .badge-stable {
        background-color: #064e3b;
        color: #34d399;
        padding: 3px 10px;
        border-radius: 6px;
        font-weight: 600;
        font-size: 0.78rem;
    }
    .badge-unstable {
        background-color: #7f1d1d;
        color: #f87171;
        padding: 3px 10px;
        border-radius: 6px;
        font-weight: 600;
        font-size: 0.78rem;
    }
</style>
""", unsafe_allow_html=True)



# ---------------------------------------------------------
# Model Loading & Caching
# ---------------------------------------------------------
@st.cache_resource(show_spinner=False)
def load_prediction_model():
    """Loads the trained Random Forest classifier or trains from raw data if absent."""
    if MODEL_PATH.exists():
        return joblib.load(MODEL_PATH)
    model, _ = train_model()
    MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, MODEL_PATH)
    return model
```

```
@st.cache_data
def load_comparison_metrics():
    """Loads empirical model evaluation metrics."""
    if METRICS_PATH.exists():
        return pd.read_csv(METRICS_PATH)
    return None



@st.cache_data
def load_sample_dataset():
    """Loads raw dataset for distribution and testing."""
    if DATA_PATH.exists():
        return pd.read_csv(DATA_PATH)
    return None



@st.cache_data
def load_test_split():
    """Returns the 20% held-out test split that the app model never trained on."""
    _, X_test, _, y_test = load_and_split_data(DATA_PATH)
    return X_test.assign(**{TARGET_COLUMN: y_test})



# Load Model
model = load_prediction_model()
metrics_df = load_comparison_metrics()
raw_data_df = load_sample_dataset()


# ------------------------------------------------------------
# Preset Scenarios for Instant Testing
# ------------------------------------------------------------
SCENARIO_PRESETS = {
    "Select a Scenario Preset...": None,
    "🟢 Resilient Grid (Stable)": {
        "tau1": 1.25, "tau2": 1.40, "tau3": 1.15, "tau4": 1.30,
        "p1": 3.60, "p2": -1.20, "p3": -1.20, "p4": -1.20,
        "g1": 0.15, "g2": 0.20, "g3": 0.18, "g4": 0.22,
        "desc": "Synchronized and resilient. Low reaction times (fast damping) and minimal price sensitivity ensure
rapid stabilization under perturbations."
    },
    "🔴 High-Latency Grid (Blackout Risk)": {
        "tau1": 8.75, "tau2": 9.20, "tau3": 8.90, "tau4": 9.50,
        "p1": 4.50, "p2": -1.80, "p3": -1.50, "p4": -1.20,
        "g1": 0.85, "g2": 0.92, "g3": 0.88, "g4": 0.90,
        "desc": "Extremely sluggish participant reaction times coupled with aggressive price elasticity lead to
phase angle divergence."
    },
    "🟡 Marginal Oscillation (Boundary)": {
        "tau1": 5.10, "tau2": 4.80, "tau3": 5.40, "tau4": 4.90,
        "p1": 3.75, "p2": -1.25, "p3": -1.25, "p4": -1.25,
        "g1": 0.50, "g2": 0.55, "g3": 0.48, "g4": 0.52,
        "desc": "Operating right at the critical boundary of Lyapunov exponent stability."
    },
    "⚡ Heavy Generator Stress": {
        "tau1": 2.10, "tau2": 6.80, "tau3": 3.40, "tau4": 7.10,
        "p1": 5.20, "p2": -1.95, "p3": -1.75, "p4": -1.50,
        "g1": 0.30, "g2": 0.75, "g3": 0.40, "g4": 0.80,
        "desc": "Extreme power generation from the central supplier node with heterogeneous consumer response
latencies."
    }
}

# ------------------------------------------------------------
# Sidebar Controls & Navigation
# ------------------------------------------------------------
with st.sidebar:
    st.markdown("""
    <div style="display:flex; align-items:center; gap:12px; margin-bottom:14px;">
```

```
        <span style="font-size:2.2rem;">⚡</span>
        <div>
            <h3 class="brand-title">GridStability AI</h3>
            <span class="brand-subtitle">AUTONOMOUS ML SYSTEM</span>
        </div>
    </div>
    """, unsafe_allow_html=True)


    st.markdown("---")
    st.subheader("🎯 Scenario Quick Presets")
    st.caption("Load pre-configured power grid operating states:")
    selected_preset_key = st.selectbox(
        "Choose test scenario",
        options=list(SCENARIO_PRESETS.keys()),
        index=0,
        label_visibility="collapsed"
    )

    preset_data = SCENARIO_PRESETS[selected_preset_key]
    if preset_data:
        st.info(f"💡 **Scenario Overview:** {preset_data['desc']}")


    st.markdown("---")
    st.subheader("⚙️ Simulation Settings")
    auto_balance_power = st.checkbox(
        "⚡ Auto-Balance Generator",
        value=True,
        help="Automatically computes p1 = -(p2 + p3 + p4) to guarantee zero net power imbalance."
    )



# ------------------------------------------------------------
# Top Hero Banner
# ------------------------------------------------------------
st.markdown("""
<div class="hero-container">
    <div class="hero-badge">⚡ Autonomous Telemetry & Diagnostics AI</div>
    <h1 class="hero-title">Decentralized Electrical Grid Stability Predictor</h1>
    <p class="hero-subtitle">
        Real-time machine learning inference for a 4-node smart electricity network.
        Detects oscillatory divergence, phase instability, and blackout risks from dynamic participant reaction
times, power flows, and price elasticity.
    </p>
</div>
""", unsafe_allow_html=True)


# ------------------------------------------------------------
# KPI Metrics Row
# ------------------------------------------------------------
kpi_col1, kpi_col2, kpi_col3, kpi_col4 = st.columns(4)


with kpi_col1:
    st.markdown("""
    <div class="kpi-card">
        <div class="kpi-label">Active Classifier</div>
        <div class="kpi-value">Random Forest</div>
        <div class="kpi-caption">200 Decision Estimators</div>
    </div>
    """, unsafe_allow_html=True)


with kpi_col2:
    st.markdown("""
    <div class="kpi-card">
        <div class="kpi-label">Test Set Accuracy</div>
        <div class="kpi-value" style="color:#10b981;">92.90%</div>
        <div class="kpi-caption">+29.10% over Baseline</div>
    </div>
    """, unsafe_allow_html=True)


with kpi_col3:
```

```
    st.markdown("""
    <div class="kpi-card">
        <div class="kpi-label">Macro F1-Score</div>
        <div class="kpi-value" style="color:#38bdf8;">0.9220</div>
        <div class="kpi-caption">Harmonic Mean of Classes</div>
    </div>
    """, unsafe_allow_html=True)

with kpi_col4:
    st.markdown("""
    <div class="kpi-card">
        <div class="kpi-label">Target Leakage Risk</div>
        <div class="kpi-value" style="color:#a855f7;">0.00%</div>
        <div class="kpi-caption">Eigenvalue (stab) Excluded</div>
    </div>
    """, unsafe_allow_html=True)

st.write("")

# ------------------------------------------------------------
# Main Tabs Navigation
# ------------------------------------------------------------
tab_sim, tab_analytics, tab_topology, tab_batch = st.tabs([
    "🕹️ Interactive Grid Simulator",
    "📊 Model Performance & Benchmarking",
    "⚛️ 4-Node Grid Topology & Theory",
    "📁 Batch Simulation & CSV Lab"
])

# ============================================================
# TAB 1: INTERACTIVE SIMULATOR
# ============================================================
with tab_sim:
    st.markdown("### 🎛️ Network Parameter Configuration")
    st.caption("Adjust the operating parameters of the decentralized 4-node power grid or load a scenario from the
sidebar.")

    # Preset values fallback helper
    def get_val(key, default):
        if preset_data and key in preset_data:
            return float(preset_data[key])
        return float(default)

    sim_form = st.container()

    with sim_form:
        param_col1, param_col2, param_col3 = st.columns([1, 1, 1])

        # Column 1: Reaction Times
        with param_col1:
            st.markdown("""
            <div class="glass-card">
                <div class="glass-card-header">
                    <span>⏱️</span> Participant Latency
                </div>
            """, unsafe_allow_html=True)
            st.caption("Reaction time in seconds (0.5s – 10.0s). Higher values slow down damping.")

            tau1 = st.slider("Supplier Reaction (tau1)", 0.5, 10.0, get_val("tau1", 2.93), 0.05, help="Central
Power Generating Plant")
            tau2 = st.slider("Consumer 1 Reaction (tau2)", 0.5, 10.0, get_val("tau2", 3.05), 0.05)
            tau3 = st.slider("Consumer 2 Reaction (tau3)", 0.5, 10.0, get_val("tau3", 4.20), 0.05)
            tau4 = st.slider("Consumer 3 Reaction (tau4)", 0.5, 10.0, get_val("tau4", 3.85), 0.05)
            st.markdown("</div>", unsafe_allow_html=True)

        # Column 2: Power Flows
        with param_col2:
            st.markdown("""
            <div class="glass-card">
```

```
            <div class="glass-card-header">
                <span> ⚡ </span> Power Balance
            </div>
        """, unsafe_allow_html=True)
        st.caption("Nominal power in p.u. Rule: Supplier p1 = -(p2 + p3 + p4).")

        p2 = st.slider("Consumer 1 Demand (p2)", -2.00, -0.50, get_val("p2", -1.25), 0.01)
        p3 = st.slider("Consumer 2 Demand (p3)", -2.00, -0.50, get_val("p3", -1.25), 0.01)
        p4 = st.slider("Consumer 3 Demand (p4)", -2.00, -0.50, get_val("p4", -1.25), 0.01)

        auto_p1_value = -round(p2 + p3 + p4, 4)

        if auto_balance_power:
            p1 = auto_p1_value
            st.number_input("Supplier Generation (p1) [Auto-Balanced]", value=float(p1), disabled=True)
        else:
            p1 = st.number_input("Supplier Generation (p1)", min_value=1.50, max_value=6.00,
value=get_val("p1", 3.75), step=0.01)

        # Net Balance Alert
        net_power = p1 + p2 + p3 + p4
        if abs(net_power) < 0.05:
            st.markdown(f"<span style='color:#10b981; font-weight:600; font-size:0.85rem;'>✅ Power Balanced:
Net ΔP = {net_power:+.3f} p.u.</span>", unsafe_allow_html=True)
        else:
            st.markdown(f"<span style='color:#ef4444; font-weight:600; font-size:0.85rem;'>⚠️ Power Imbalance
Alert: Net ΔP = {net_power:+.3f} p.u.</span>", unsafe_allow_html=True)

        st.markdown("</div>", unsafe_allow_html=True)

    # Column 3: Price Elasticity
    with param_col3:
        st.markdown("""
        <div class="glass-card">
            <div class="glass-card-header">
                <span>📈</span> Price Elasticity
            </div>
        """, unsafe_allow_html=True)
        st.caption("Willingness to adapt consumption to price shifts (0.05 – 1.0).")

        g1 = st.slider("Supplier Elasticity (g1)", 0.05, 1.00, get_val("g1", 0.50), 0.01)
        g2 = st.slider("Consumer 1 Elasticity (g2)", 0.05, 1.00, get_val("g2", 0.45), 0.01)
        g3 = st.slider("Consumer 2 Elasticity (g3)", 0.05, 1.00, get_val("g3", 0.55), 0.01)
        g4 = st.slider("Consumer 3 Elasticity (g4)", 0.05, 1.00, get_val("g4", 0.60), 0.01)
        st.markdown("</div>", unsafe_allow_html=True)

# ------------------------------------------------------
# Real-time Prediction Execution
# ------------------------------------------------------
current_features = [tau1, tau2, tau3, tau4, p1, p2, p3, p4, g1, g2, g3, g4]
input_df = pd.DataFrame([current_features], columns=FEATURE_COLUMNS)

prediction = model.predict(input_df)[0]
probabilities = model.predict_proba(input_df)[0]
prob_dict = dict(zip(model.classes_, probabilities))
prob_stable = prob_dict["stable"]
prob_unstable = prob_dict["unstable"]

st.markdown("---")
st.markdown("### 🔬 Real-Time Stability Diagnosis")

res_col1, res_col2 = st.columns([1, 1.2])

with res_col1:
    # Verdict Box
    if prediction == "stable":
        st.markdown(f"""
        <div class="verdict-box verdict-stable">
```

```
                <div class="verdict-tag" style="color:#a7f3d0;">DIAGNOSTIC STATUS</div>
                <h2 class="verdict-title">STABLE OPERATING STATE</h2>
                <div style="font-size:1.05rem; color:#d1fae5; margin-top:8px;">
                    Grid parameters are within the safe differential damping basin.
                </div>
                <div style="display:flex; justify-content:center; gap:20px; margin-top:16px;">
                    <div>
                        <div style="font-size:0.8rem; color:#a7f3d0; text-transform:uppercase;">Stable
Likelihood</div>
                        <div style="font-size:1.8rem; font-weight:800; color:#ffffff;">{prob_stable:.1%}</div>
                    </div>
                    <div style="border-left:1px solid rgba(255,255,255,0.2); padding-left:20px;">
                        <div style="font-size:0.8rem; color:#a7f3d0; text-transform:uppercase;">Blackout Risk</div>
                        <div style="font-size:1.8rem; font-weight:800; color:#fca5a5;">{prob_unstable:.1%}</div>
                    </div>
                </div>
            </div>
            """, unsafe_allow_html=True)
        else:
            st.markdown(f"""
            <div class="verdict-box verdict-unstable">
                <div class="verdict-tag" style="color:#fecaca;">DIAGNOSTIC STATUS</div>
                <h2 class="verdict-title">UNSTABLE (CASCADE RISK)</h2>
                <div style="font-size:1.05rem; color:#fee2e2; margin-top:8px;">
                    Oscillatory divergence detected. High probability of synchronous trip.
                </div>
                <div style="display:flex; justify-content:center; gap:20px; margin-top:16px;">
                    <div>
                        <div style="font-size:0.8rem; color:#fecaca; text-transform:uppercase;">Unstable
Probability</div>
                        <div style="font-size:1.8rem; font-weight:800; color:#ffffff;">{prob_unstable:.1%}</div>
                    </div>
                    <div style="border-left:1px solid rgba(255,255,255,0.2); padding-left:20px;">
                        <div style="font-size:0.8rem; color:#fecaca; text-transform:uppercase;">Stable
Likelihood</div>
                        <div style="font-size:1.8rem; font-weight:800; color:#a7f3d0;">{prob_stable:.1%}</div>
                    </div>
                </div>
            </div>
            """, unsafe_allow_html=True)


        # Gauge Speedometer Chart
        fig_gauge = go.Figure(go.Indicator(
            mode="gauge+number",
            value=prob_stable * 100,
            domain={'x': [0, 1], 'y': [0, 1]},
            title={'text': "<b>Stability Safety Index (%)</b>", 'font': {'size': 18, 'color': '#f8fafc'}},
            number={'suffix': "%", 'font': {'size': 32, 'color': '#ffffff'}},
            gauge={
                'axis': {'range': [0, 100], 'tickwidth': 1, 'tickcolor': "#94a3b8"},
                'bar': {'color': "#10b981" if prediction == "stable" else "#ef4444", 'thickness': 0.28},
                'bgcolor': "rgba(30, 41, 59, 0.4)",
                'borderwidth': 1,
                'bordercolor': "rgba(255,255,255,0.1)",
                'steps': [
                    {'range': [0, 45], 'color': 'rgba(239, 68, 68, 0.3)'},
                    {'range': [45, 60], 'color': 'rgba(245, 158, 11, 0.3)'},
                    {'range': [60, 100], 'color': 'rgba(16, 185, 129, 0.3)'}
                ],
                'threshold': {
                    'line': {'color': "#38bdf8", 'width': 3},
                    'thickness': 0.8,
                    'value': 50
                }
            }
        ))
        fig_gauge.update_layout(
            paper_bgcolor='rgba(0,0,0,0)',
            plot_bgcolor='rgba(0,0,0,0)',
            font={'color': "#f8fafc", 'family': "Outfit"},
            height=260,
            margin=dict(l=20, r=20, t=50, b=20)
        )
        st.plotly_chart(fig_gauge, use_container_width=True)
```

```
    with res_col2:
        # Radar Chart of 12 Features
        categories = ["tau1", "tau2", "tau3", "tau4", "p1", "p2 (abs)", "p3 (abs)", "p4 (abs)", "g1", "g2", "g3",
"g4"]
        # Normalize features approximately for radar display:
        radar_values = [
            tau1 / 10.0, tau2 / 10.0, tau3 / 10.0, tau4 / 10.0,
            p1 / 6.0, abs(p2) / 2.0, abs(p3) / 2.0, abs(p4) / 2.0,
            g1, g2, g3, g4
        ]

        fig_radar = go.Figure()
        fig_radar.add_trace(go.Scatterpolar(
            r=radar_values,
            theta=categories,
            fill='toself',
            name='Current Operating Vector',
            fillcolor='rgba(56, 189, 248, 0.25)',
            line=dict(color='#38bdf8', width=2)
        ))
        # Ideal reference baseline
        baseline_stable = [0.25, 0.25, 0.25, 0.25, 0.60, 0.60, 0.60, 0.60, 0.30, 0.30, 0.30, 0.30]
        fig_radar.add_trace(go.Scatterpolar(
            r=baseline_stable,
            theta=categories,
            name='Typical Resilient Grid Profile',
            line=dict(color='#10b981', dash='dot', width=1.5)
        ))

        fig_radar.update_layout(
            polar=dict(
                radialaxis=dict(visible=True, range=[0, 1], gridcolor='rgba(255,255,255,0.1)',
tickfont=dict(color='#64748b')),
                angularaxis=dict(gridcolor='rgba(255,255,255,0.1)', tickfont=dict(color='#cbd5e1'))
            ),
            showlegend=True,
            legend=dict(orientation="h", yanchor="bottom", y=-0.2, xanchor="center", x=0.5,
font=dict(color='#cbd5e1')),
            paper_bgcolor='rgba(0,0,0,0)',
            plot_bgcolor='rgba(0,0,0,0)',
            height=370,
            margin=dict(l=40, r=40, t=20, b=40)
        )
        st.markdown("##### 🔬 Multi-Dimensional Parameter Fingerprint")
        st.plotly_chart(fig_radar, use_container_width=True)


        # Automated Engineering Advisory
        st.markdown("""
        <div style="background:rgba(30, 41, 59, 0.5); border-left:4px solid #38bdf8; border-radius:8px;
padding:12px 16px; margin-top:10px;">
            <div style="font-size:0.85rem; font-weight:700; color:#38bdf8;">⚡ AUTOMATED DISPATCH ADVISORY</div>
            <div style="font-size:0.8rem; color:#cbd5e1; margin-top:4px;">
        """, unsafe_allow_html=True)
        if tau1 > 6.0 or tau2 > 6.0 or tau3 > 6.0 or tau4 > 6.0:
            st.markdown("• **High Participant Latency Warning:** Participant reaction time $\\tau$ exceeds 6.0s.
Grid frequency controller may fail to dampen oscillation.")
        if g1 > 0.7 or g2 > 0.7 or g3 > 0.7 or g4 > 0.7:
            st.markdown("• **High Elasticity Volatility:** Participant sensitivity $\\gamma > 0.70$ causes rapid
demand fluctuations following price signals.")
        if abs(net_power) > 0.05:
            st.markdown("• **Unbalanced System:** Net load mismatch $\\sum p \\neq 0$. Ensure generation equals
total consumer absorption.")
        if prediction == "stable" and max(tau1, tau2, tau3, tau4) <= 4.0:
            st.markdown("• **Optimal Damping State:** System operates within robust Lyapunov stability margins.")
        st.markdown("</div></div>", unsafe_allow_html=True)



# ============================================================
# TAB 2: MODEL BENCHMARKING & ANALYTICS
# ============================================================
with tab_analytics:
```

```
    st.markdown("### ⏳ Rigorous Machine Learning Model Comparison")
    st.caption("All models evaluated strictly on the exact same 2,000 held-out test cases (80/20 Stratified Split,
`random_state=42`).")

    col_bench1, col_bench2 = st.columns([1.1, 1])

    with col_bench1:
        st.markdown("##### 📋 Quantitative Metric Scorecard")
        if metrics_df is not None:
            # Set Metric as index so only numeric columns are styled and compared
            display_metrics = metrics_df.set_index("Metric")
            st.dataframe(
                display_metrics.style.format("{:.4f}").highlight_max(axis=1, color="#1e3a8a"),
                use_container_width=True,
                height=420
            )
        else:
            st.warning("Metrics file outputs/metrics/model_comparison.csv not found.")

    with col_bench2:
        st.markdown("##### 🌲 Random Forest Feature Importance (MDI)")
        if hasattr(model, "feature_importances_"):
            fi_df = pd.DataFrame({
                "Parameter": FEATURE_COLUMNS,
                "Importance": model.feature_importances_
            }).sort_values(by="Importance", ascending=True)

            fig_fi = px.bar(
                fi_df,
                x="Importance",
                y="Parameter",
                orientation='h',
                color="Importance",
                color_continuous_scale="Tealgrn",
                text_auto='.3f'
            )
            fig_fi.update_layout(
                paper_bgcolor='rgba(0,0,0,0)',
                plot_bgcolor='rgba(0,0,0,0)',
                font={'color': "#f8fafc"},
                xaxis=dict(gridcolor='rgba(255,255,255,0.08)'),
                yaxis=dict(gridcolor='rgba(255,255,255,0.08)'),
                height=400,
                margin=dict(l=20, r=20, t=10, b=20)
            )
            st.plotly_chart(fig_fi, use_container_width=True)

    st.markdown("---")
    st.markdown("### 🛡️ Critical Safety Risk: Analysis of 'False Stables'")

    risk_col1, risk_col2 = st.columns([1, 1])
    with risk_col1:
        st.markdown("""
        <div class="glass-card">
            <h4 style="color:#ef4444; margin-top:0;">⚠️ The High Cost of False Negatives in Power Grids</h4>
            <p style="font-size:0.9rem; color:#cbd5e1; line-height:1.6;">
                In high-voltage power transmission, predicting that an unstable grid is <b>"Stable" (False
Stable)</b> is catastrophic. Operators fail to dispatch emergency spinning reserves or island the sub-network,
triggering <b>cascading system-wide blackouts</b>.
            </p>
            <ul style="font-size:0.88rem; color:#94a3b8;">
                <li><b>Logistic Regression:</b> 146 False Stables (Unstable Recall: 88.56%)</li>
                <li><b>Random Forest:</b> <span style="color:#10b981; font-weight:700;">Only 48 False Stables
(Unstable Recall: 96.24%)</span></li>
            </ul>
        </div>
        """, unsafe_allow_html=True)

    with risk_col2:
        st.markdown("""
        <div class="glass-card">
```

```
            <h4 style="color:#38bdf8; margin-top:0;">🔍 Why Random Forest Outperformed Linear Classification</h4>
            <p style="font-size:0.9rem; color:#cbd5e1; line-height:1.6;">
                Electrical grid dynamics are dictated by coupled nonlinear differential equations. Stability is not
a linear hyperplane separation; it depends on <b>complex interaction terms</b> (e.g., reaction time $\\tau_i$
compounded with high elasticity $\\gamma_i$).
            </p>
            <p style="font-size:0.88rem; color:#94a3b8;">
                Random Forest's 200 orthogonal decision surfaces easily capture non-convex stability boundaries
that linear models misclassify.
            </p>
        </div>
        """, unsafe_allow_html=True)



# ============================================================
# TAB 3: 4-NODE TOPOLOGY & THEORY
# ============================================================
with tab_topology:
    st.markdown("### ⚛️ 4-Node Smart Grid Star Network Topology")
    st.caption("Decentralized power grid model formulated by Vadim Arzamasov et al. (2018).")

    top_col1, top_col2 = st.columns([1.2, 1])

    with top_col1:
        st.markdown("""
        ```mermaid
        graph TD
            N1["⚡ Node 1 (Central Supplier)<br>Power: p1 > 0<br>Latency: tau1 | Elasticity: g1"]
            N2["🏭 Node 2 (Consumer 1)<br>Power: p2 < 0<br>Latency: tau2 | Elasticity: g2"]
            N3["🏢 Node 3 (Consumer 2)<br>Power: p3 < 0<br>Latency: tau3 | Elasticity: g3"]
            N4["🏠 Node 4 (Consumer 3)<br>Power: p4 < 0<br>Latency: tau4 | Elasticity: g4"]

            N1 <-->|Bidirectional Transmission Line| N2
            N1 <-->|Bidirectional Transmission Line| N3
            N1 <-->|Bidirectional Transmission Line| N4

            classDef gen fill:#065f46,stroke:#10b981,stroke-width:2px,color:#fff;
            classDef cons fill:#1e293b,stroke:#38bdf8,stroke-width:2px,color:#fff;
            class N1 gen;
            class N2,N3,N4 cons;
        ```
        """)

    with top_col2:
        st.markdown(r"""
        <div class="glass-card">
            <div class="glass-card-header">📐 Mathematical Governing Equations</div>
            <p style="font-size:0.85rem; color:#cbd5e1; line-height:1.6;">
                The smart grid stability problem is modeled using swing equations for rotatory generators and
delayed consumer price adaptation:
            </p>
            <div style="background:rgba(0,0,0,0.3); padding:10px 14px; border-radius:8px; font-family:'JetBrains
Mono'; font-size:0.85rem; color:#38bdf8; margin:10px 0;">
                M_i \ddot{\theta}_i + D_i \dot{\theta}_i = P_i - \sum K_{ij} \sin(\theta_i - \theta_j)
            </div>
            <p style="font-size:0.82rem; color:#94a3b8; line-height:1.5;">
                • τ_i: Reaction delay of participants adjusting demand to market signals.
                • p_i: Injected or extracted active power satisfying $\sum p_i = 0$.
                • γ_i: Price elasticity governing economic feedback strength.
                • Eigenvalue Root ($stab$): Max real part of root. If $stab \le 0 \implies Stable$, else
$Unstable$.
            </p>
        </div>
        """, unsafe_allow_html=True)

==
TAB 4: BATCH SIMULATION & CSV LAB
==
with tab_batch:
```

```
    st.markdown("### 📁 Batch Testing & Simulation Laboratory")
    st.caption("Upload a CSV with the 12 operating parameters or run a quick batch on 20 random samples from the
test corpus.")


    batch_mode = st.radio("Select batch input source:", ["Use 20 Random Test Records from Dataset", "Upload Custom
CSV File"], horizontal=True)


    if batch_mode == "Use 20 Random Test Records from Dataset":
        if DATA_PATH.exists():
            # Sample only from the held-out test split so the accuracy shown is unbiased
            sample_batch = load_test_split().sample(20, random_state=42)
            X_batch = sample_batch[FEATURE_COLUMNS]
            ground_truth = sample_batch[TARGET_COLUMN].values


            batch_preds = model.predict(X_batch)
            batch_probs = model.predict_proba(X_batch)[:, 1]


            results_df = X_batch.copy()
            results_df["Actual Class"] = ground_truth
            results_df["Predicted Class"] = batch_preds
            results_df["Unstable Confidence"] = [f"{p:.1%}" for p in batch_probs]
            results_df["Correct?"] = results_df["Actual Class"] == results_df["Predicted Class"]


            style_fn = results_df.style.map if hasattr(results_df.style, "map") else results_df.style.applymap
            styled_batch = style_fn(
                lambda val: 'background-color: rgba(16, 185, 129, 0.2); color:#10b981; font-weight:700;' if val is
True else ('background-color: rgba(239, 68, 68, 0.2); color:#ef4444; font-weight:700;' if val is False else ''),
                subset=['Correct?']
            )
            st.dataframe(styled_batch, use_container_width=True)


            acc_batch = (batch_preds == ground_truth).mean()
            st.success(f"Batch Accuracy on 20 Held-Out Test Cases: **{acc_batch:.1%}** ({(batch_preds ==
ground_truth).sum()} / 20 correct)")
        else:
            st.error("Raw dataset not found.")


    else:
        uploaded_file = st.file_uploader("Upload CSV containing the 12 operating features", type=["csv"])
        if uploaded_file is not None:
            user_df = pd.read_csv(uploaded_file)
            missing_cols = [c for c in FEATURE_COLUMNS if c not in user_df.columns]
            if missing_cols:
                st.error(f"Missing required columns in CSV: {missing_cols}")
            else:
                user_preds = model.predict(user_df[FEATURE_COLUMNS])
                user_probs = model.predict_proba(user_df[FEATURE_COLUMNS])[:, 1]
                user_df["Predicted_Stability"] = user_preds
                user_df["Unstable_Probability"] = user_probs
                st.dataframe(user_df, use_container_width=True)


                csv_download = user_df.to_csv(index=False).encode('utf-8')
                st.download_button(
                    label="📥 Download Predictions CSV",
                    data=csv_download,
                    file_name="grid_stability_predictions.csv",
                    mime="text/csv",
                )

# ----------------------------------------------------------
# Footer
# ----------------------------------------------------------
st.markdown("---")
st.markdown("""
<div style="display:flex; justify-content:space-between; align-items:center; color:#64748b; font-size:0.8rem;">
    <div>⚡ <b>Smart Electrical Grid Stability AI Platform</b></div>
    <div>Real-time Diagnostics & Non-Linear Machine Learning</div>
</div>
""", unsafe_allow_html=True)
```

A.10 requirements.txt

```
# Versions used to produce the reported results (Python 3.14, Windows 11)
pandas==3.0.5
numpy==2.5.3
matplotlib==3.11.1
seaborn==0.13.2
scikit-learn==1.9.1
joblib==1.6.0
jupyter
nbformat==5.11.1
python-docx==1.2.0


# Interactive prediction UI (app.py)
streamlit>=1.30.0
plotly>=5.18.0
```

A.11 notebooks/01_dataset_audit.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path


# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
df = pd.read_csv(DATA_PATH)
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
df.head()


# --- Cell 3 ---
df.dtypes.to_frame("dtype").T


# --- Cell 4 ---
print("Missing values per column:")
print(df.isna().sum().to_frame("missing").T)
print("\nExact duplicate rows:", df.duplicated().sum())
print("Duplicate feature vectors (ignoring labels):", df.drop(columns=["stab", "stabf"]).duplicated().sum())


# --- Cell 5 ---
from src.data_checks import validity_checks
validity_checks(df)


# --- Cell 6 ---
counts = df["stabf"].value_counts()
pd.DataFrame({"count": counts, "percent": (counts / len(df) * 100).round(2)})


# --- Cell 7 ---
df.describe().T.round(3)
```

A.12 notebooks/02_exploratory_data_analysis.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path


# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
df = pd.read_csv(DATA_PATH)
FEATURES = [c for c in df.columns if c not in ("stab", "stabf")]
PALETTE = {"unstable": "#e74c3c", "stable": "#2ecc71"}


fig, ax = plt.subplots(figsize=(6, 4))
sns.countplot(data=df, x="stabf", hue="stabf", order=["unstable", "stable"], palette=PALETTE, legend=False, ax=ax)
for c in ax.containers:
    ax.bar_label(c, fmt="%d")
ax.set_title("Distribution of Stability Class (stabf)")
plt.show()


# --- Cell 3 ---
fig, ax = plt.subplots(figsize=(6, 4))
sns.boxplot(data=df, x="stabf", y="stab", hue="stabf", order=["unstable", "stable"], palette=PALETTE, legend=False,
ax=ax)
ax.axhline(0, color="black", ls="--", alpha=0.7)
ax.set_title("stab vs stabf — the label is a threshold of stab")
plt.show()
df.groupby("stabf")["stab"].agg(["min", "max", "mean"])


# --- Cell 4 ---
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(df[FEATURES].corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1, ax=ax)
ax.set_title("Correlation Heatmap of 12 Operating Parameters")
plt.show()


# --- Cell 5 ---
long_df = df.melt(id_vars="stabf", value_vars=FEATURES, var_name="feature")
g = sns.FacetGrid(long_df, col="feature", col_wrap=4, hue="stabf", sharex=False, sharey=False,
                  palette=PALETTE, height=2.2, aspect=1.4)
g.map(sns.kdeplot, "value", fill=True, alpha=0.35)
g.add_legend(title="stabf")
plt.show()


# --- Cell 6 ---
# Mean of each feature per class and the standardised difference between classes
summary = df.groupby("stabf")[FEATURES].mean().T
summary["diff / std"] = (summary["unstable"] - summary["stable"]) / df[FEATURES].std()
summary.round(3).sort_values("diff / std", key=abs, ascending=False)
```

A.13 notebooks/03_preprocessing_and_split.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path
```

```
# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
from src.preprocessing import FEATURE_COLUMNS, TARGET_COLUMN, load_and_split_data


print("Features (X):", FEATURE_COLUMNS)
print("Target (y):", TARGET_COLUMN)
print("Excluded: 'stab' (target leakage — stabf is derived from it)")


X_train, X_test, y_train, y_test = load_and_split_data(DATA_PATH, test_size=0.20, random_state=42)
print("\nTrain:", X_train.shape, " Test:", X_test.shape)
pd.DataFrame({"train %": y_train.value_counts(normalize=True) * 100,
              "test %": y_test.value_counts(normalize=True) * 100}).round(2)


# --- Cell 3 ---
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler


pca = PCA().fit(StandardScaler().fit_transform(X_train))
pca_df = pd.DataFrame({"component": range(1, 13),
                       "explained_variance_ratio": pca.explained_variance_ratio_,
                       "cumulative": pca.explained_variance_ratio_.cumsum()}).round(4)
fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(pca_df["component"], pca_df["explained_variance_ratio"], label="Per component")
ax.plot(pca_df["component"], pca_df["cumulative"], "o-", color="tab:orange", label="Cumulative")
ax.set_xticks(range(1, 13)); ax.set_xlabel("Principal component"); ax.legend()
ax.set_title("PCA explained variance (standardised training features)")
plt.show()
pca_df
```

A.14 notebooks/04_logistic_regression.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path

# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
from sklearn.model_selection import StratifiedKFold, cross_validate
from src.preprocessing import FEATURE_COLUMNS, load_and_split_data
from src.train_models import get_logistic_regression_pipeline
```

```
from src.evaluation import evaluate_predictions

X_train, X_test, y_train, y_test = load_and_split_data(DATA_PATH)
lr = get_logistic_regression_pipeline()
lr


# --- Cell 3 ---
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_res = cross_validate(lr, X_train, y_train, cv=cv, scoring=["accuracy", "f1_macro"], return_train_score=True)
pd.DataFrame(cv_res).drop(columns=["fit_time", "score_time"]).agg(["mean", "std"]).round(4)


# --- Cell 4 ---
lr.fit(X_train, y_train)
res = evaluate_predictions(y_test, lr.predict(X_test))
print(res["classification_report"])
print("Confusion matrix (rows = actual, cols = predicted; order: stable, unstable)")
print(res["confusion_matrix"])


# --- Cell 5 ---
coef = pd.Series(lr.named_steps["classifier"].coef_[0], index=FEATURE_COLUMNS)
coef.sort_values().plot.barh(figsize=(7, 4), title="Standardised LR coefficients (positive -> more likely
unstable)")
plt.axvline(0, color="black", lw=0.8); plt.show()
coef.round(3).sort_values()
```

A.15 notebooks/05_random_forest.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path


# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
from sklearn.metrics import accuracy_score
from sklearn.model_selection import StratifiedKFold, cross_validate
from src.preprocessing import FEATURE_COLUMNS, load_and_split_data
from src.train_models import get_random_forest_model
from src.evaluation import evaluate_predictions


X_train, X_test, y_train, y_test = load_and_split_data(DATA_PATH)
rf = get_random_forest_model()
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_res = cross_validate(rf, X_train, y_train, cv=cv, scoring=["accuracy", "f1_macro"], return_train_score=True,
n_jobs=-1)
pd.DataFrame(cv_res).drop(columns=["fit_time", "score_time"]).agg(["mean", "std"]).round(4)


# --- Cell 3 ---
rf.fit(X_train, y_train)
print("Training accuracy:", accuracy_score(y_train, rf.predict(X_train)))
res = evaluate_predictions(y_test, rf.predict(X_test))
print("Test accuracy:", round(res["accuracy"], 4))
print(res["classification_report"])
print("Confusion matrix (order: stable, unstable)")
print(res["confusion_matrix"])
```

```
# --- Cell 4 ---
mdi = pd.Series(rf.feature_importances_, index=FEATURE_COLUMNS).sort_values()
mdi.plot.barh(figsize=(7, 4), title="Random Forest impurity-based importance (MDI)")
plt.show()
mdi.sort_values(ascending=False).round(4)
```

A.16 notebooks/06_model_comparison.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path


# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"


import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline


# --- Cell 2 ---
from sklearn.metrics import ConfusionMatrixDisplay, roc_auc_score, roc_curve
from src.preprocessing import load_and_split_data
from src.train_models import get_logistic_regression_pipeline, get_random_forest_model
from src.evaluation import evaluate_predictions


X_train, X_test, y_train, y_test = load_and_split_data(DATA_PATH)
models = {"Logistic Regression": get_logistic_regression_pipeline().fit(X_train, y_train),
          "Random Forest": get_random_forest_model().fit(X_train, y_train)}
results = {name: evaluate_predictions(y_test, m.predict(X_test)) for name, m in models.items()}
baseline = evaluate_predictions(y_test, np.full(len(y_test), "unstable"))


keys = ["accuracy", "macro_f1", "weighted_f1", "stable_precision", "stable_recall", "stable_f1",
        "unstable_precision", "unstable_recall", "unstable_f1"]
table = pd.DataFrame({"Majority baseline": {k: baseline[k] for k in keys},
                      **{n: {k: r[k] for k in keys} for n, r in results.items()}})
for name, r in {"Majority baseline": baseline, **results}.items():
    cm = r["confusion_matrix"]
    table.loc["false_stable (missed unstable)", name] = cm[1, 0]
    table.loc["false_unstable (false alarm)", name] = cm[0, 1]
table.round(4)


# --- Cell 3 ---
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, (name, r), cmap in zip(axes, results.items(), ["Blues", "Greens"]):
    ConfusionMatrixDisplay(r["confusion_matrix"], display_labels=["stable", "unstable"]).plot(ax=ax, cmap=cmap,
values_format="d")
    ax.set_title(f"{name} (accuracy {r['accuracy']:.3f})")
plt.tight_layout(); plt.show()


# --- Cell 4 ---
fig, ax = plt.subplots(figsize=(6, 5))
for name, m in models.items():
    p = m.predict_proba(X_test)[:, list(m.classes_).index("unstable")]
    fpr, tpr, _ = roc_curve(y_test == "unstable", p)
    ax.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc_score(y_test == 'unstable', p):.3f})")
ax.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate (unstable recall)")
ax.set_title("ROC curves — positive class = unstable"); ax.legend(); plt.show()
```

A.17 notebooks/07_improvement_experiments.ipynb (code cells)

```
# --- Cell 1 ---
import sys
from pathlib import Path

# Make the project's src/ package importable when running from notebooks/
PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "electrical_grid_stability.csv"

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.width", 140)
%matplotlib inline

# --- Cell 2 ---
from src.experiments import run_experiments
R = run_experiments()

# --- Cell 3 ---
R["cv"].T

# --- Cell 4 ---
R["ablation"].pivot(index="Feature set", columns="Model", values="cv_macro_f1")

# --- Cell 5 ---
print("Best parameters:", R["best_params"])
R["tuning"].groupby("Model").head(5)

# --- Cell 6 ---
from IPython.display import Image
print("Chosen thresholds (>= 98% unstable recall on training folds):", R["chosen_thresholds"])
Image(str(PROJECT_ROOT / "outputs" / "figures" / "threshold_tradeoff.png"), width=900)

# --- Cell 7 ---
R["final"]

# --- Cell 8 ---
Image(str(PROJECT_ROOT / "outputs" / "figures" / "roc_curves.png"), width=600)

# --- Cell 9 ---
display(R["permutation"].pivot(index="feature", columns="Model", values="importance_mean")
        .sort_values("Random Forest", ascending=False).round(4))
Image(str(PROJECT_ROOT / "outputs" / "figures" / "permutation_importance.png"), width=750)
```